



Sistema de Monitoreo: Acciones Peligrosas en el Conductor

Iván Augusto Camargo López
Santiago Torres Barbosa

2230033
2202024

 Mediapipe

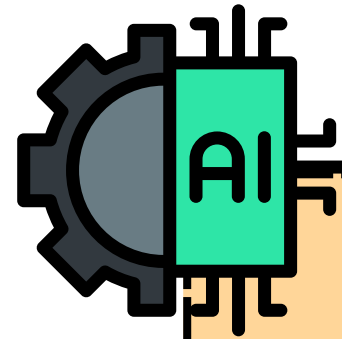
 ViTPose

Índice

1. Objetivos
2. Motivación del Problema
3. Dataset Empleado y Preprocesamiento
4. Análisis Estadístico - Gráficas Relevantes
5. Procesamiento para ML
6. Aplicación: Algoritmos de Machine Learning
7. Top Mejores Resultados
8. Buscando relaciones para mejorar el Modelo.
9. Reducción de Dimensiones: PCA/TSNE
10. Reducción de Dimensiones: Otros Algoritmos
11. Métodos de Agrupamiento: K-Means, DBSCAN y Aglomerative Clustering
12. Otros algoritmos de clustering
13. Resultados (videos)
14. Limitaciones y Trabajo Futuro
15. Conclusiones Finales
16. Repositorio
17. Gracias



Objetivo



Integrar herramientas de Computer Vision (modelos de extracción de posturas como Mediapipe FaceMesh) y métodos tradicionales de Machine Learning para la detección de acciones peligrosas realizadas al volante, como microsueños, a partir de la clasificación de cuadros individuales provenientes de videos del rostro del conductor. Esto con el fin de alertar al conductor y pasajeros de vehículo, y evitar accidentes viales potencialmente fatales.



Motivación del Problema



- La fatiga y la distracción al volante son factores críticos en accidentes de tráfico.
- Conductas como la somnolencia afectan la concentración del conductor y aumentan el riesgo de colisión.
- Este proyecto utiliza MediaPipe y ViTPose para analizar imágenes y videos y detectar signos de distracción o fatiga en conductores.
- En el futuro, se espera implementar la detección en tiempo real para mejorar la seguridad vial.

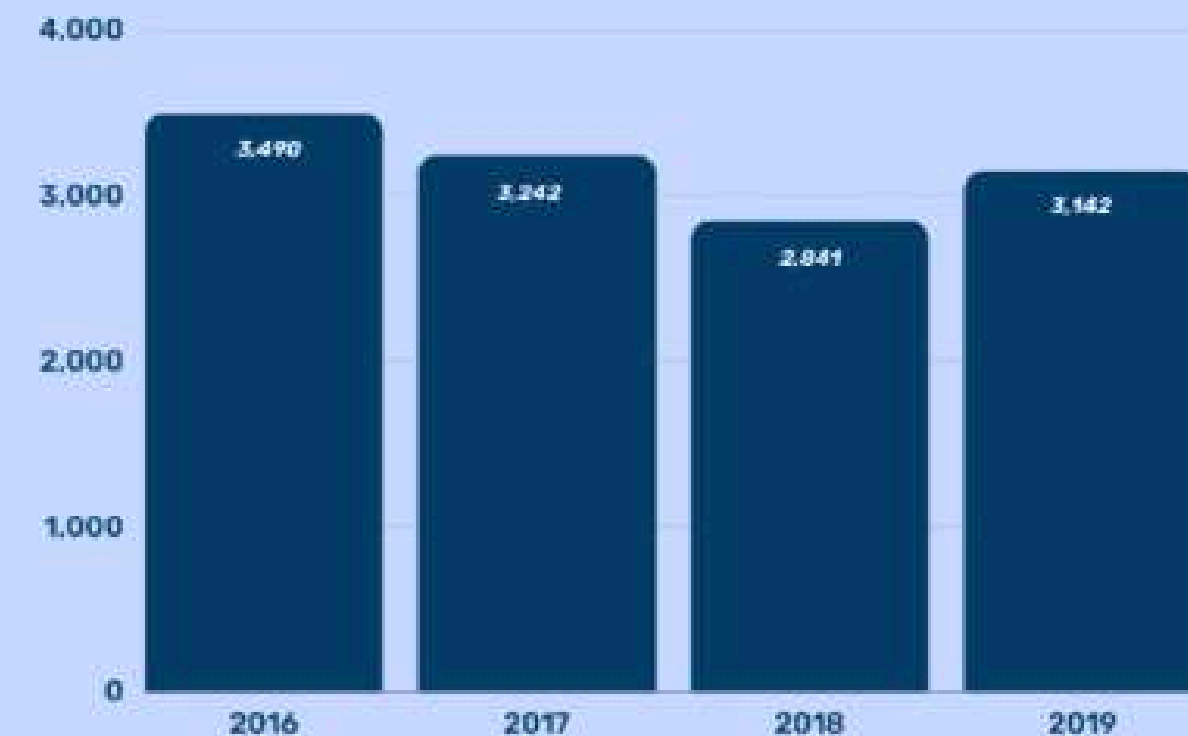
Estadísticas Relevantes

Fatiga y somnolencia al volante:

- En 2017, se estimó que 91,000 accidentes reportados por la policía involucraron a conductores somnolientos, resultando en aproximadamente 50,000 lesiones y casi 800 muertes. [Manejar Soñoliento | NHTSA](#)
- En Colombia, los micro sueños están vinculados a un alto porcentaje de siniestros viales.
([Microsueños y siniestros en Colombia: Una realidad preocupante - ÁRTIMO](#))

Fatalidades Causadas por Conductores Distráidos

2016 - 2019



3,000

El número de personas que fallecen cada año en accidentes de tráfico relacionados con conductores distraídos

Fuente: Centros para el Control y Prevención de Enfermedades



Dataset Empleado - DMD

- El **Driver Monitoring Dataset** destaca por ser uno de los datasets de monitoreo de conductor más variados y exhaustivos disponibles públicamente.

- El dataset, en su conjunto, suma más de **25 TBs de datos**, distribuidos en videos clasificados cuidadosamente de acuerdo a las acciones realizadas en cada video.

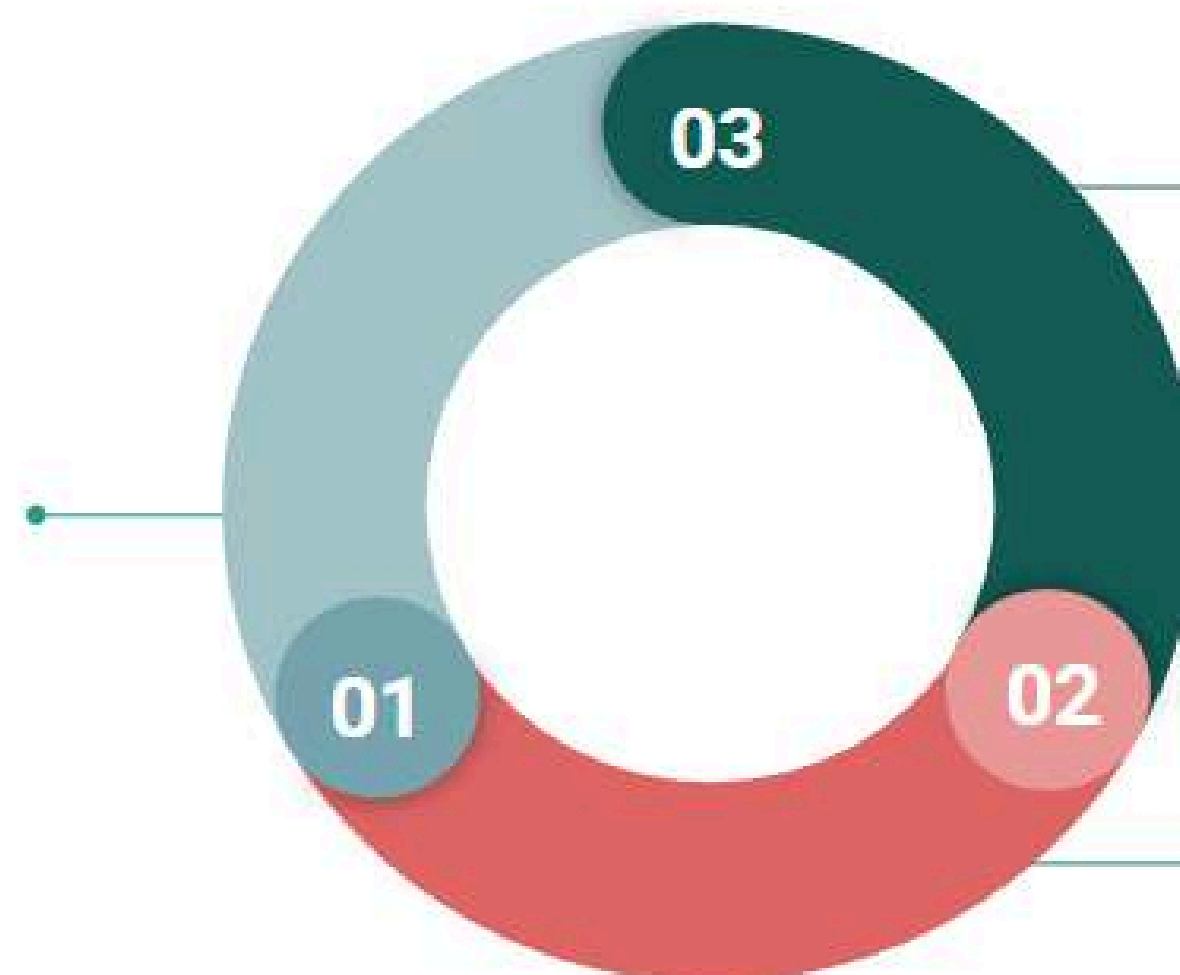


vicomtech

MEMBER OF BASQUE RESEARCH
& TECHNOLOGY ALLIANCE

Distribución del Dataset

Gaze & Hands
Orientación de la visión del conductor: distintos lados del vehículo. Posición de las manos en relación al volante



Drowsiness

Signos de fatiga. Bostezar, microsueños, frecuencia de parpadeos, estado de los ojos

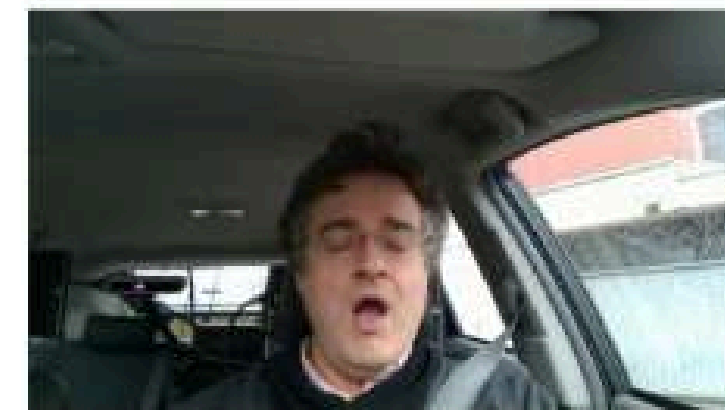
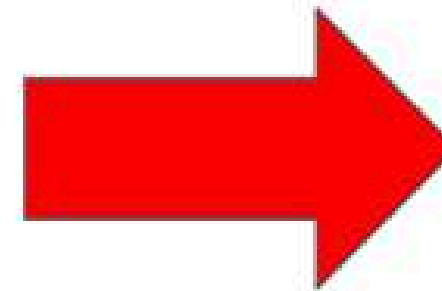
Distracción*

Hablar al teléfono, beber, usar el radio, ponerse maquillaje, hablar con los pasajeros

¿Cómo procesamos los datos?

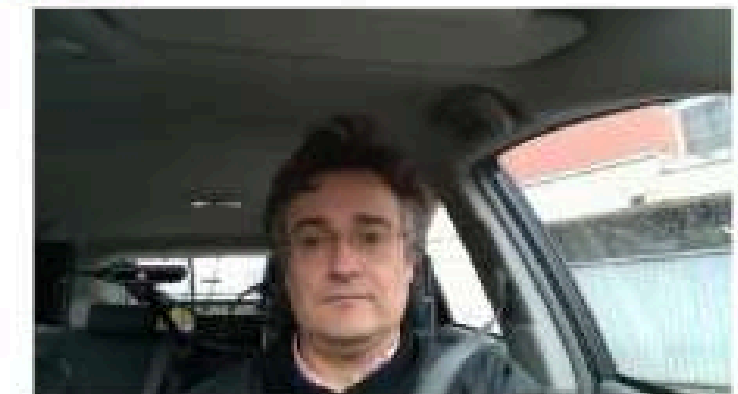
- El dataset se encuentra etiquetado a través de un modelo llamado OpenLabel

```
OpenLabel {
  "dataset": {
    "frames": [
      {
        "start": 0.0,
        "end": 1.0,
        "label": "eyes/eyes_open"
      },
      {
        "start": 1.0,
        "end": 2.0,
        "label": "yawning/yawning_without_hand"
      },
      {
        "start": 2.0,
        "end": 3.0,
        "label": "blinks/blinking"
      }
    ]
  }
}
```

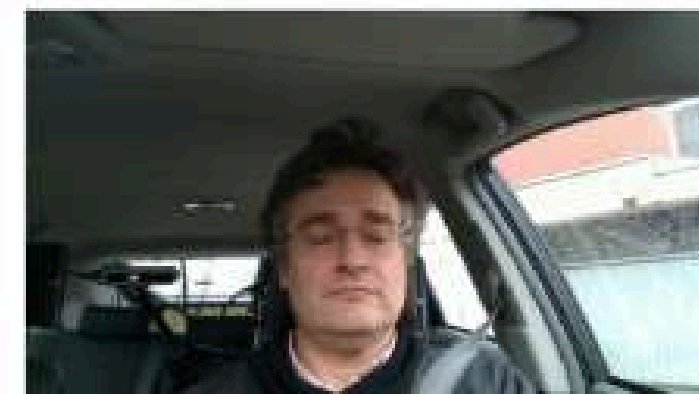


yawning/yawning_without_hand

eyes/eyes_open

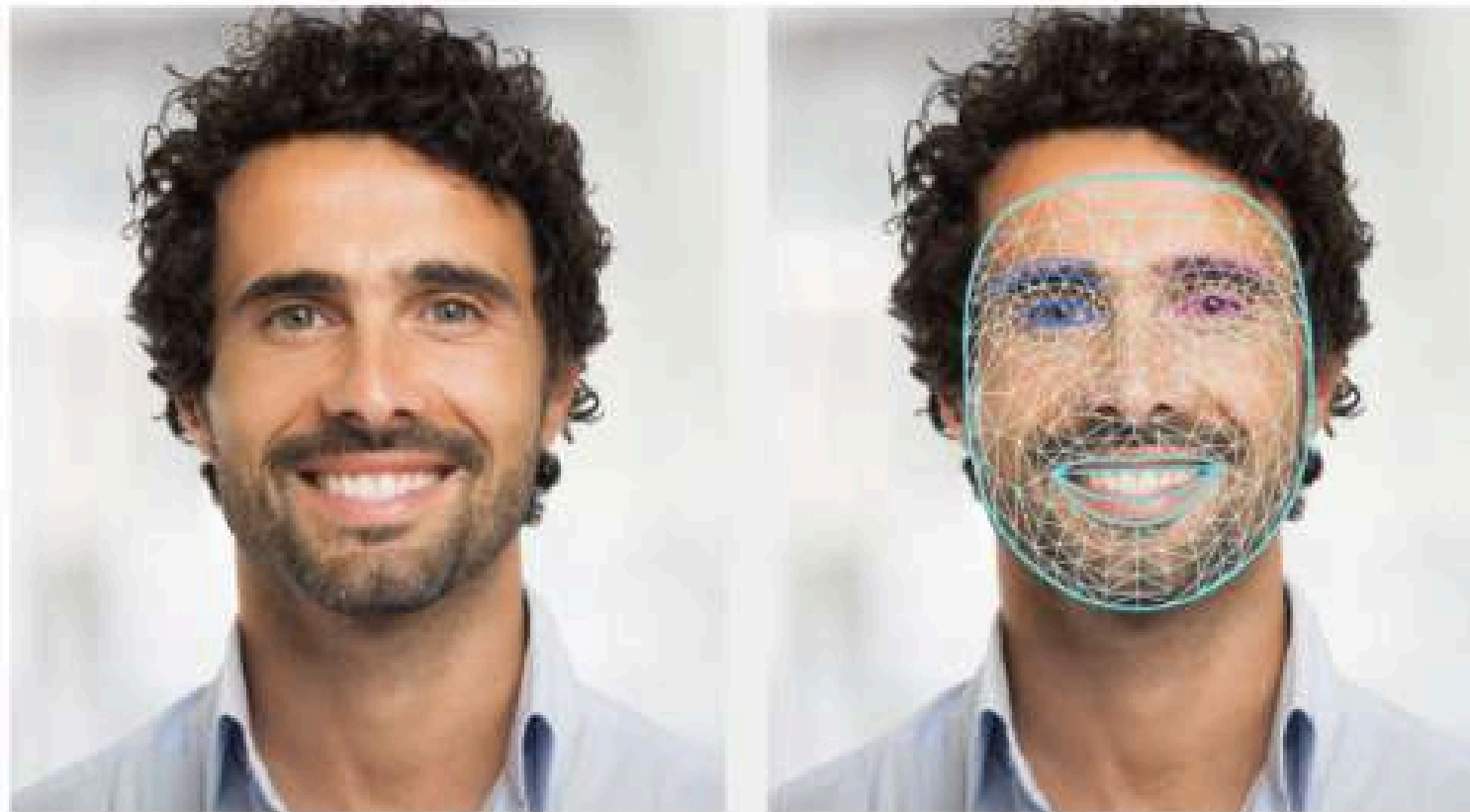


blinks/blinking



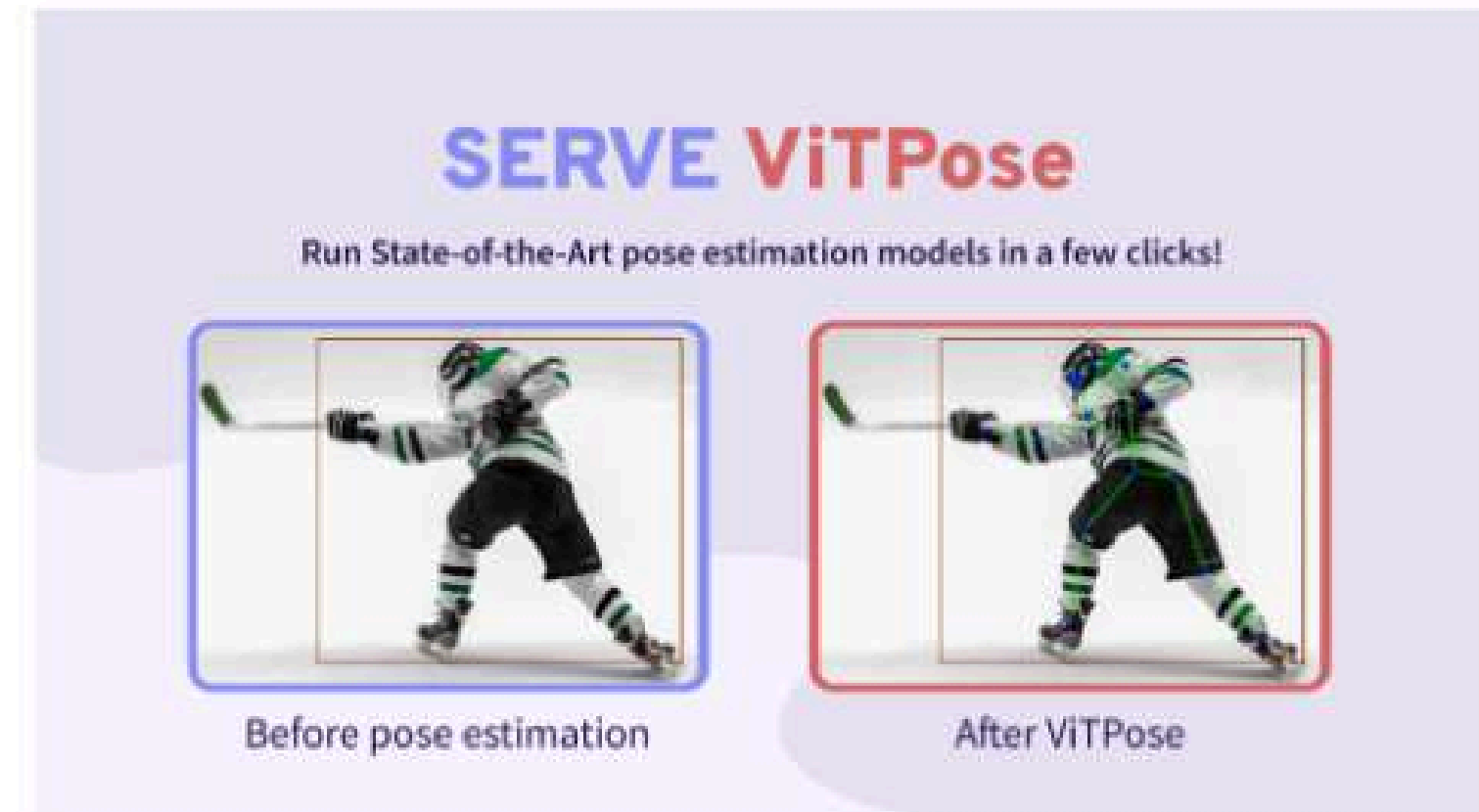
¿Cómo procesamos los datos?

- Vamos a extraer **landmarks** de los sujetos. Es decir, coordenadas de puntos asociados a sus posturas



MediaPipe

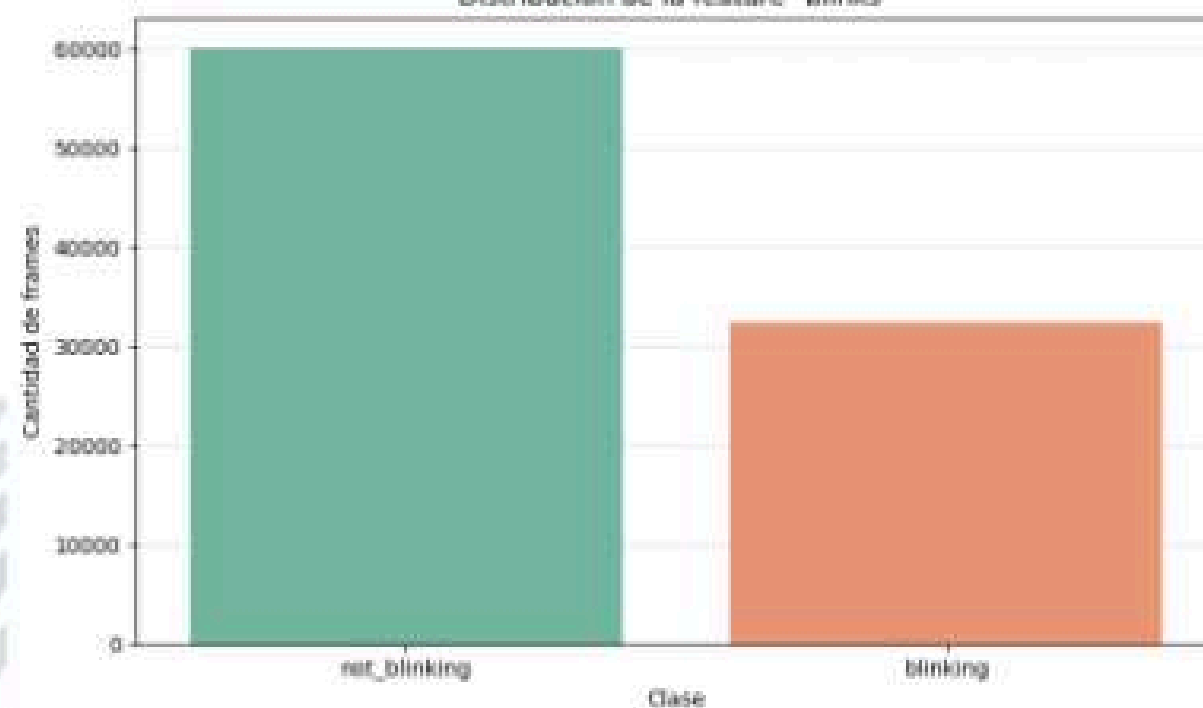
- Así, **evitamos trabajar con imágenes completas**, y obtenemos muchas 'columas' con las que trabajar.



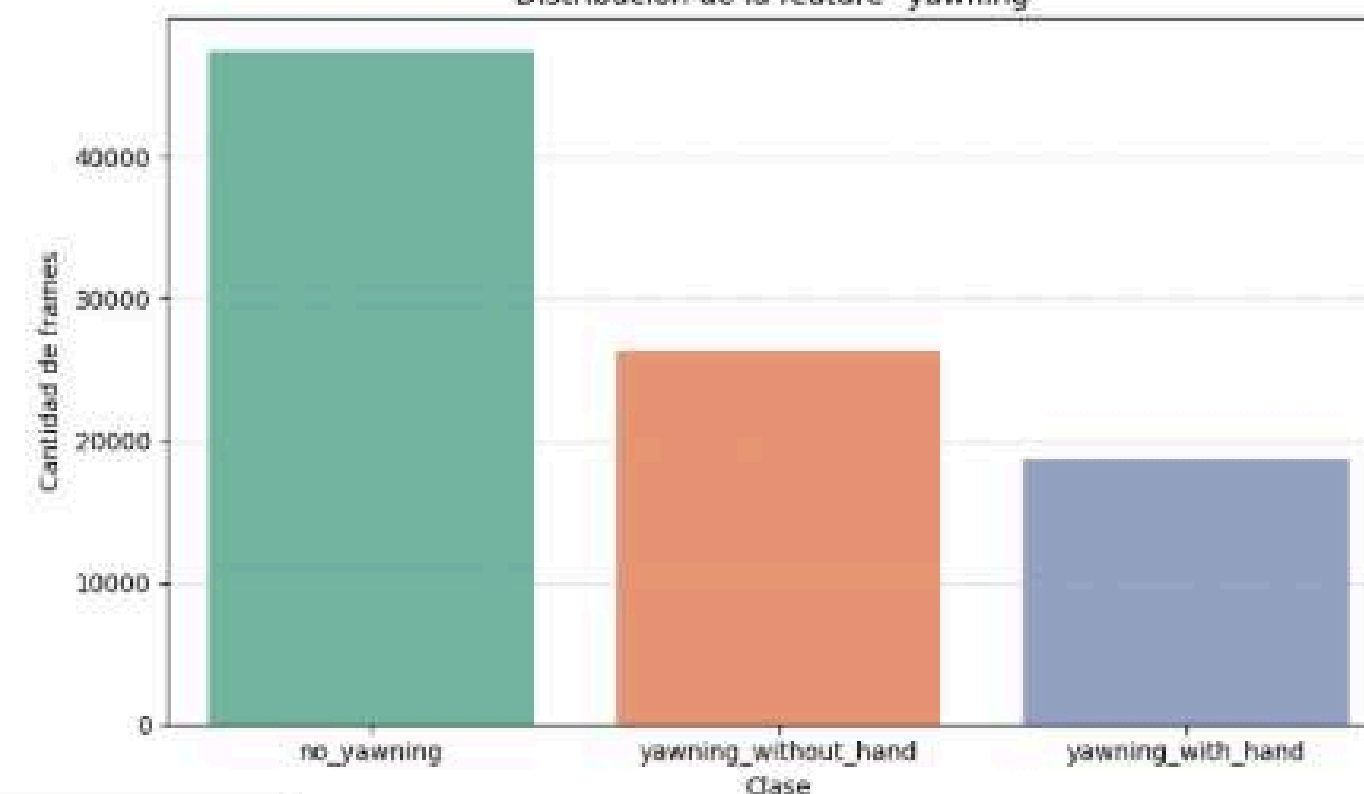
ViTPose

Análisis Estadístico - Gráficas Relevantes

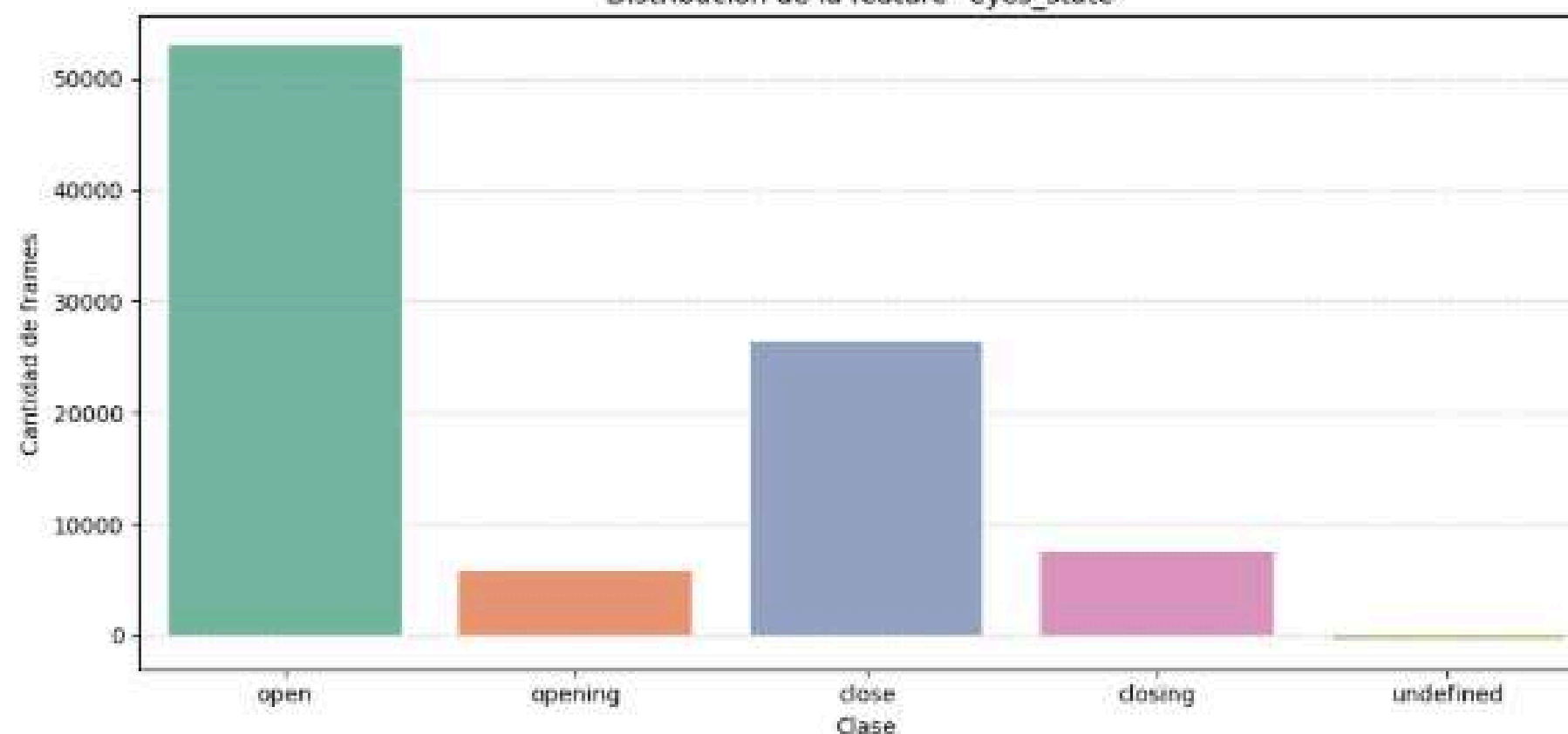
Distribución de la feature "blinks"

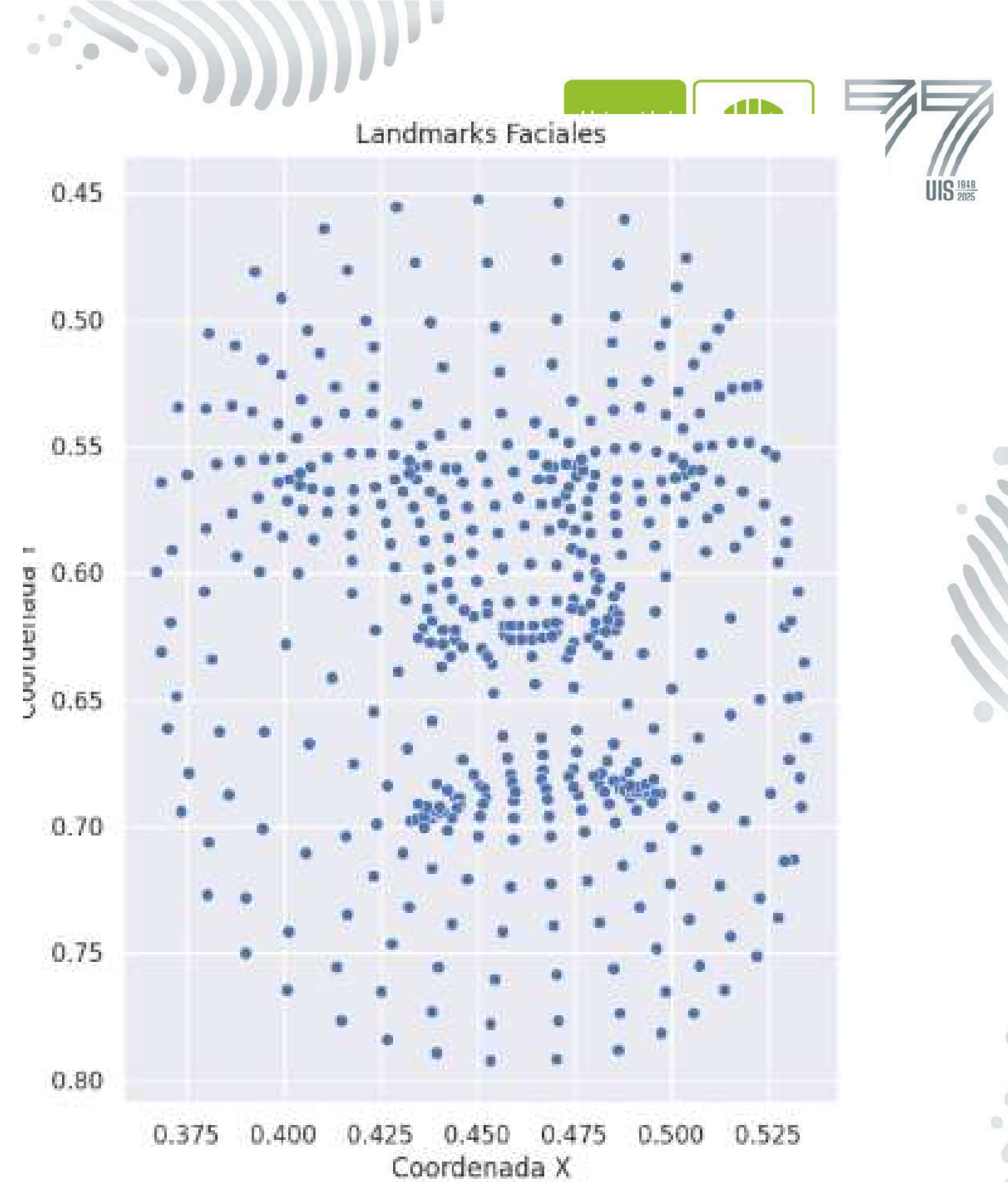
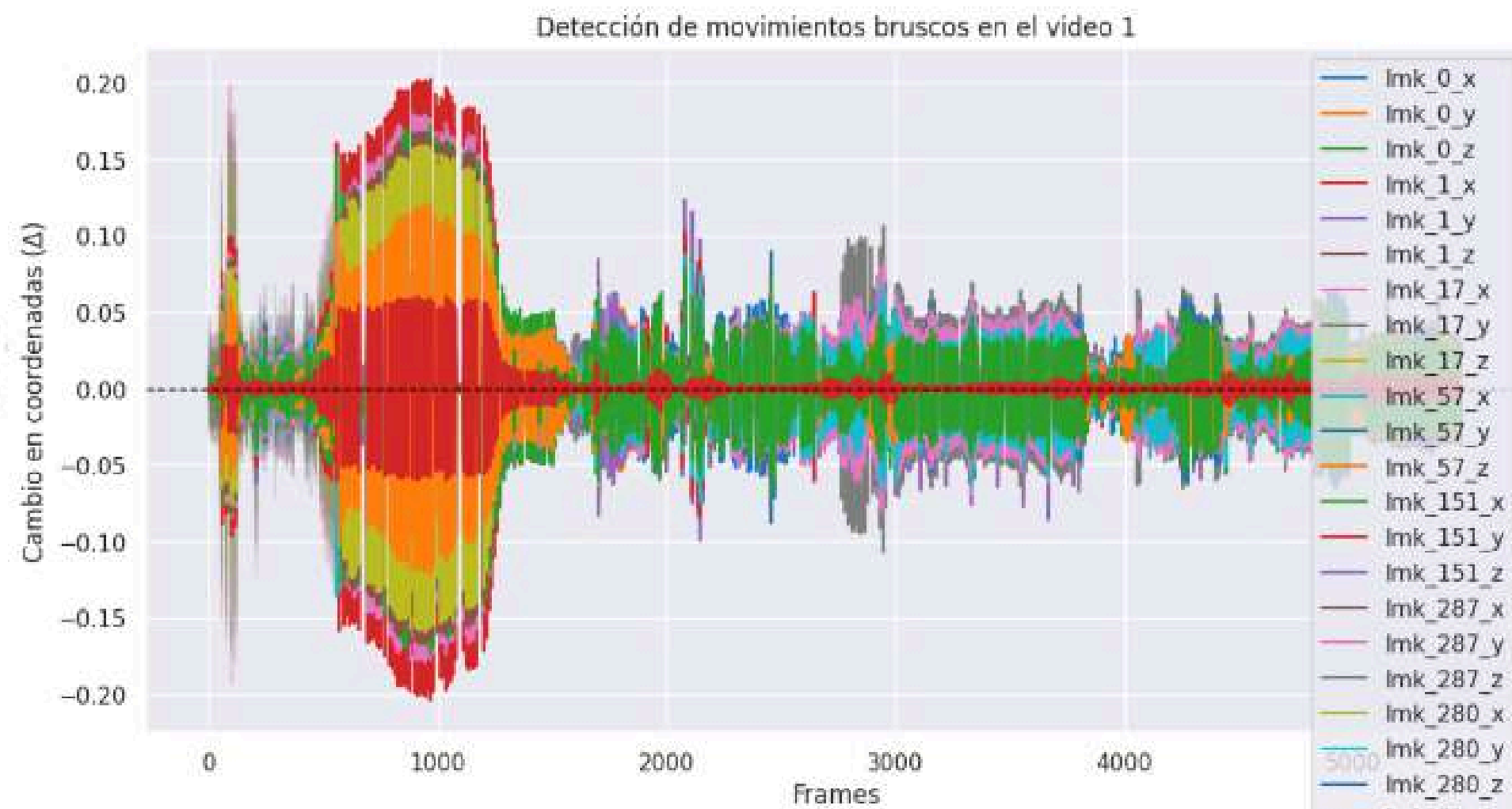
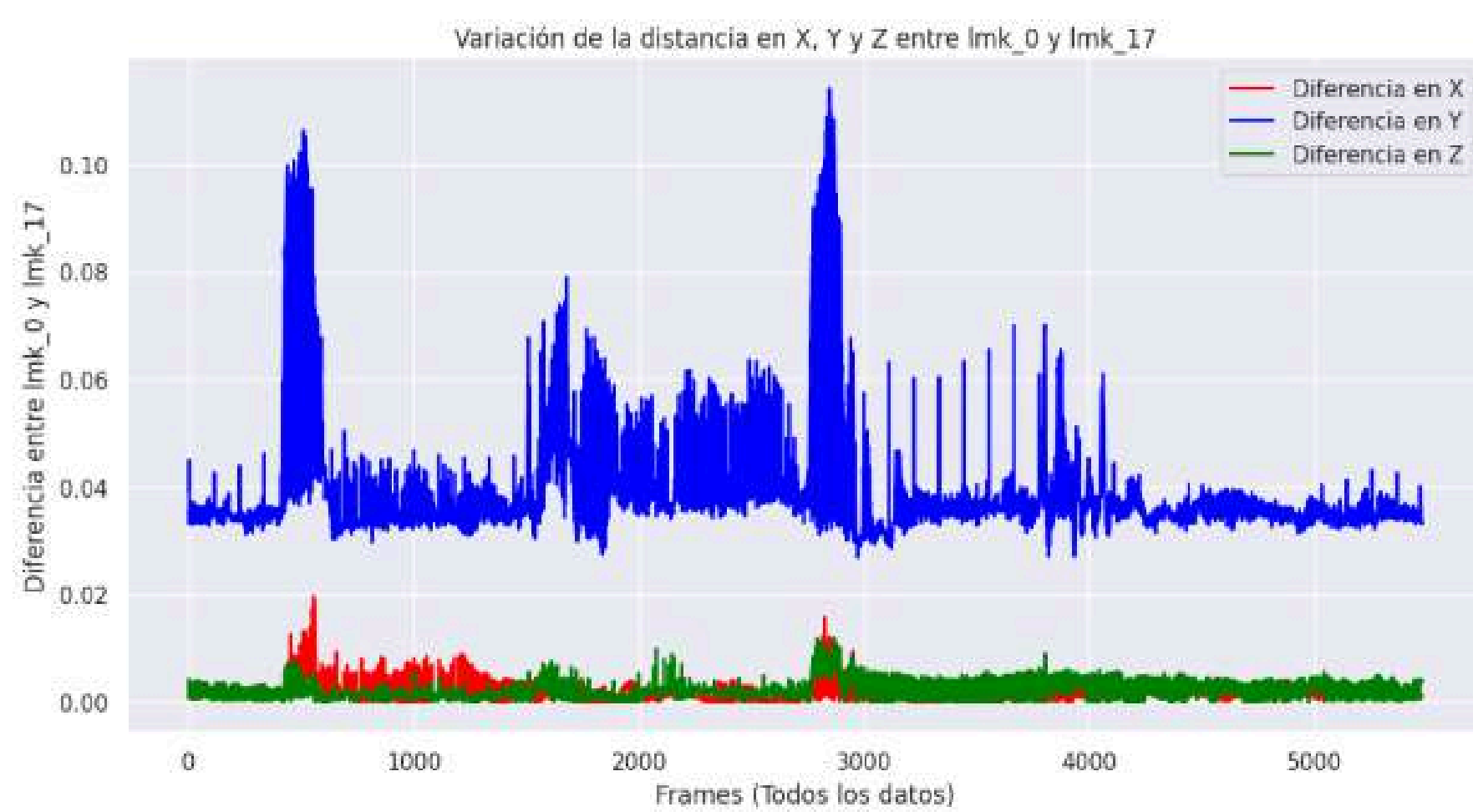


Distribución de la feature "yawning"



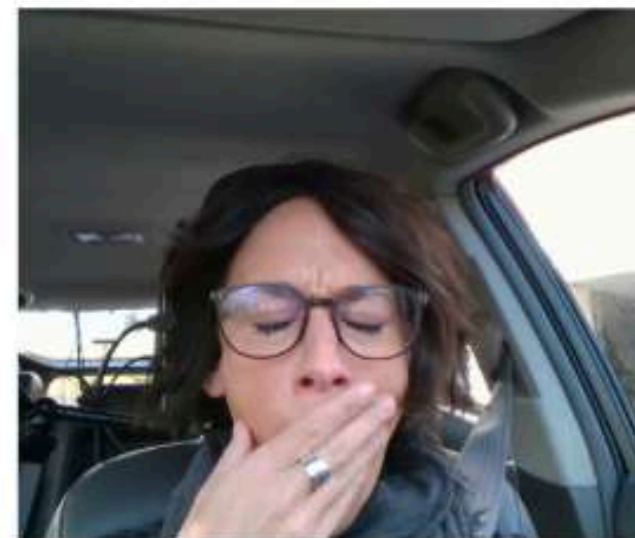
Distribución de la feature "eyes_state"





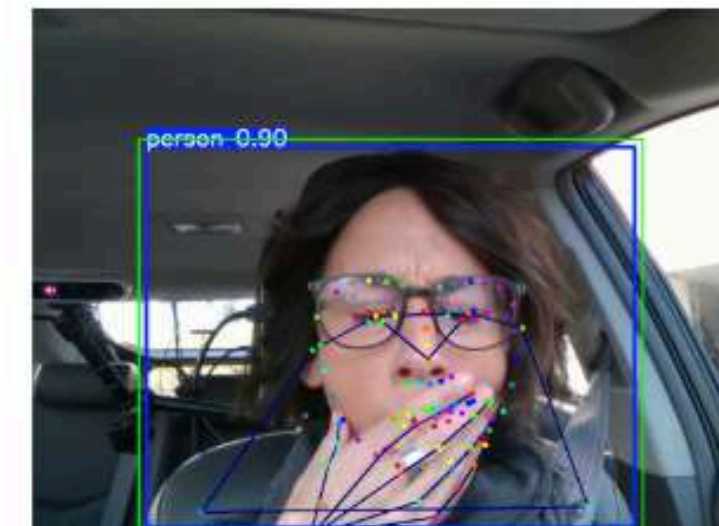
Procesamiento para ML

- A través de **ViTPose** y **Mediapipe** extraemos **landmarks** de puntos clave en el rostro y en las manos.



- Así, no es necesario trabajar con un dataset lleno de imágenes; únicamente con un **dataframe de coordenadas**.

ViTPose



Procesamiento para ML

- Procesando a los **16 sujetos** de la partición del dataset, se termina con un total de **92491 filas** y **1453 columnas**

frame	lmk_1_x	lmk_1_y	lmk_1_z	[...]	lmk_hand_1_x	eyes_state
1	0.453872	0.192839	0.678126	[...]	0.788235	eyes_state/open
2	0.123583	0.12840	0.237893	[...]	NaN	NaN
3	[...]	[...]	[...]	[...]	[...]	eyes_state/close
[...]	[...]	[...]	[...]	[...]	[...]	eyes_state/opening

Procesamiento para ML

- Para poder ingresar procesar el dataframe con métodos de Machine Learning, realizamos un **mapeo de las etiquetas** y reemplazamos los **NaN** por **0** (simbolizando la ausencia de la coordenada)

frame	lmk_1_x	lmk_1_y	lmk_1_z	[...]	lmk_hand_1_x	eyes_state
1	0.453872	0.192839	0.678126	[...]	0.788235	1
2	0.123583	0.12840	0.237893	[...]	0	0
3	[...]	[...]	[...]	[...]	[...]	2
[...]	[...]	[...]	[...]	[...]	[...]	3

Mapeo de Etiquetas

Etiqueta Original	Clave
NaN	0
blinks/blinking	1

Etiqueta Original	Clave
NaN	0
yawning/Yawning without hand	1
yawning/Yawning with hand	2

Etiqueta Original	Clave
eyes_state/open	0
eyes_state/close	1
eyes_state/opening	2
eyes_state/closing	3
undefined	4

- Se eliminó la columna oclussion, dado que solo contenía NaN*

Aplicación: Algoritmos de Machine Learning

- Con el dataset listo y acondicionado para los algoritmos de machine learning, es el momento de **aplicar cada uno de ellos para evaluar cuál es el mejor**
- Primero, se van a utilizar los **ajustes predeterminados** de cada uno de los métodos de clasificación.



Resumen Modelos ML (test)

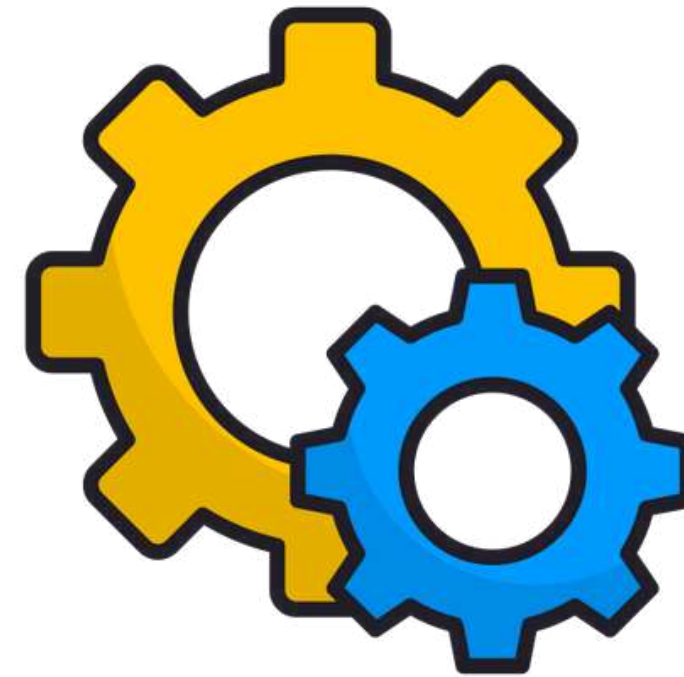
Algoritmo	Métrica	blinks	eyes_estate	yawning
Decision Tree	<u>Accuracy</u>	0.772	0.709	0.940
	<u>Recall</u>	0.772	0.709	0.940
	<u>F1_Score</u>	0.772	0.709	0.940
Naive Bayes	<u>Accuracy</u>	0.641	0.088	0.131
	<u>Recall</u>	0.641	0.088	0.131
	<u>F1_Score</u>	0.560	0.144	0.141
Random Forest	<u>Accuracy</u>	0.861	0.806	0.969
	<u>Recall</u>	0.861	0.806	0.969
	<u>F1_Score</u>	0.857	0.785	0.967

Resumen Modelos ML (test)

Algoritmo	Métrica	blinks	eyes_estate	yawning
SVM	<u>Accuracy</u>	0.658	0.653	0.878
	<u>Recall</u>	0.658	0.653	0.878
	<u>F1_Score</u>	0.532	0.515	0.821
Logistic Regression	<u>Accuracy</u>	0.658	0.652	0.878
	<u>Recall</u>	0.658	0.652	0.878
	<u>F1_Score</u>	0.535	0.516	0.822
KNN	<u>Accuracy</u>	0.843	0.789	0.969
	<u>Recall</u>	0.843	0.789	0.969
	<u>F1_Score</u>	0.842	0.773	0.968
Deep Learning	<u>Accuracy</u>	0.656	0.653	0.879
	<u>Recall</u>	0.656	0.653	0.879
	<u>F1_Score</u>	0.524	0.518	0.823

Aplicación: Mejorando Algoritmos de Machine Learning

- Con el fin de adaptar los modelos de una mejor forma al dataset, se hace necesario modificar los **hiperparámetros** inherentes a cada uno.
- Con la modificación de estos parámetros, se espera mejorar las métricas calculadas.



Resultados de Modificación de Parámetros (ejemplos)

DecisionTreeClassifier

```
max_depth=100,  
min_samples_split=100,  
min_samples_leaf=50,  
max_features='log2',  
criterion='gini',  
class_weight='balanced'
```

RandomForestClassifier

```
n_estimators=200,  
max_depth=30,  
min_samples_split=4,  
min_samples_leaf=2,  
max_features='sqrt',  
criterion='gini',  
class_weight='balanced',  
random_state=42,  
n_jobs=-1,  
oob_score=True
```

SupportVectorMachine

```
kernel='linear',  
C=2.0,  
gamma='scale',  
class_weight='balanced',  
random_state=42,  
max_iter=200,  
tol=1e-3
```

Top Mejores Resultados



RandomForestClassifier

```
max_depth=200,  
min_samples_split=2,  
min_samples_leaf=1,  
criterion='gini',  
class_weight='balanced'
```

	blinks	eyes_estate	yawning
train_Accuracy	<u>0.984</u>	<u>0.984</u>	<u>0.997</u>
test_Accuracy	<u>0.863</u>	<u>0.810</u>	<u>0.970</u>
train_Precision	<u>0.984</u>	<u>0.984</u>	<u>0.997</u>
test_Precision	<u>0.863</u>	<u>0.803</u>	<u>0.970</u>
train_Recall	<u>0.984</u>	<u>0.984</u>	<u>0.997</u>
test_Recall	<u>0.863</u>	<u>0.810</u>	<u>0.970</u>
train_F1_Score	<u>0.984</u>	<u>0.984</u>	<u>0.997</u>
test_F1_Score	<u>0.859</u>	<u>0.789</u>	<u>0.968</u>
train_Cohen_Kappa	<u>0.964</u>	<u>0.969</u>	<u>0.985</u>
test_Cohen_Kappa	<u>0.682</u>	<u>0.591</u>	<u>0.847</u>

Top Mejores Resultados



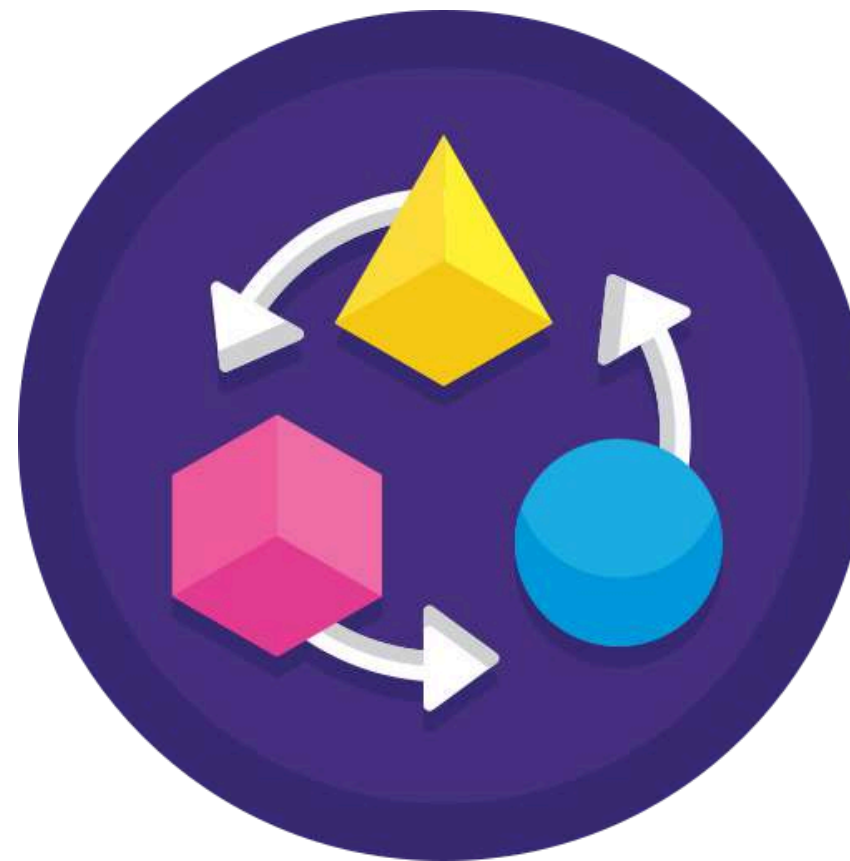
KNeighborsClassifier

n_neighbors=10,
weights='distance',
leaf_size = 50,
n_jobs=-1

	blinks	eyes_estate	yawning
train_Accuracy	<u>0.984</u>	<u>0.984</u>	<u>0.997</u>
test_Accuracy	<u>0.847</u>	<u>0.795</u>	<u>0.968</u>
train_Precision	<u>0.984</u>	<u>0.984</u>	<u>0.997</u>
test_Precision	<u>0.845</u>	<u>0.779</u>	<u>0.967</u>
train_Recall	<u>0.984</u>	<u>0.984</u>	<u>0.997</u>
test_Recall	<u>0.847</u>	<u>0.795</u>	<u>0.968</u>
train_F1_Score	<u>0.984</u>	<u>0.984</u>	<u>0.997</u>
test_F1_Score	<u>0.844</u>	<u>0.780</u>	<u>0.967</u>
train_Cohen_Kappa	<u>0.964</u>	<u>0.969</u>	<u>0.985</u>
test_Cohen_Kappa	<u>0.652</u>	<u>0.580</u>	<u>0.846</u>

Más allá: Buscando relaciones para mejorar el Modelo.

- Si bien se han logrado resultados aceptables, todavía existe un margen de mejora, especialmente para predecir el **eyes_state**



- Para ello, se puede optar por **transformar las coordenadas del dataset** nuevamente antes de ingresarlas a los métodos de Machine Learning

Vectores - Diferencia

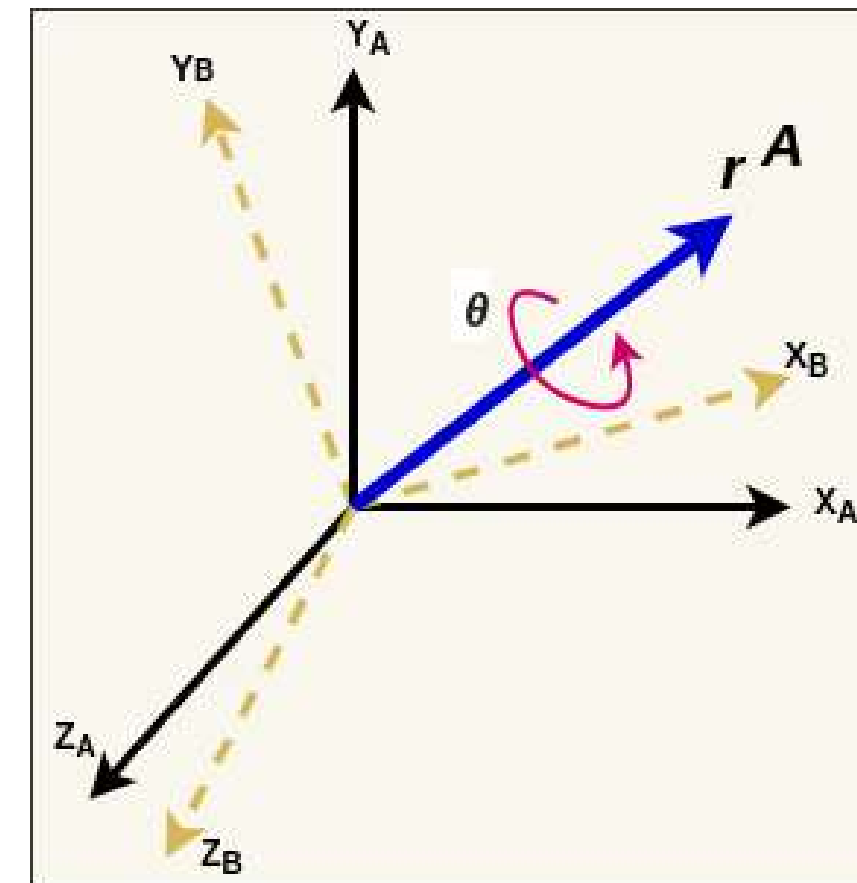
- En general, no se consiguió mejorar ninguna métrica.
- Lo más probable es que se deba a la menor cantidad de features disponibles.
- Utilizando un RandomForest:

	blinks	eyes_estate	yawning
train_Accuracy	0.975	0.975	0.996
test_Accuracy	0.809	0.757	0.944
train_Precision	0.976	0.976	0.996
test_Precision	0.810	0.753	0.945
train_Recall	0.975	0.975	0.996
test_Recall	0.809	0.757	0.944
train_F1_Score	0.975	0.975	0.996
test_F1_Score	0.799	0.718	0.938
train_Cohen_Kappa	0.944	0.952	0.980
test_Cohen_Kappa	0.545	0.437	0.693

Cuaterniones

- Un cuaternión es una extensión de los números complejos, representado como:
- Los cuaterniones representan rotaciones. Evitan problemas como el **gimbal lock**, y son herramientas matemáticas eficientes y estables
- En este caso, podemos calcular los **cuaterniones de los vectores diferencia**, con respecto a una referencia, que, en este caso, se consideró como el **eje z** ([0,0,1]).

$$q = w + xi + yj + zk$$



Cuaterniones

- En este caso, utilizando los **20 landmarks más importantes**, se obtiene un dataset con **92491 filas y 760 columnas**
- En este caso, utilizando un RandomForest, impresiona la forma en que, **con menos features, obtenemos resultados muy similares.**
- No obstante, no se superaron los récords anteriores propuestos.

	blinks	eyes_estate	yawning
train_Accuracy	0.975	0.975	0.996
test_Accuracy	0.809	0.758	0.945
train_Precision	0.976	0.976	0.996
test_Precision	0.811	0.754	0.945
train_Recall	0.975	0.975	0.996
test_Recall	0.809	0.758	0.945
train_F1_Score	0.975	0.975	0.996
test_F1_Score	0.799	0.719	0.939
train_Cohen_Kappa	0.944	0.952	0.980
test_Cohen_Kappa	0.545	0.440	0.696

División por paciente

- Sin embargo, en este caso, había un problema: **no estábamos particionando correctamente el train/test por sujetos**
- Esto llevaba a que, por ejemplo, datos del mismo sujeto estén en train y test. En otras palabras, **data leakage**.
- Hay que hacer que el modelo recuerde cómo identificar **situaciones peligrosas**, **no recordar el sujeto**
- Por eso, las nuevas particiones son de 13 sujetos para train, y 3 para test



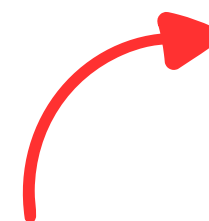
Top Mejores Resultados

(ahora sí)



RandomForestClassifier

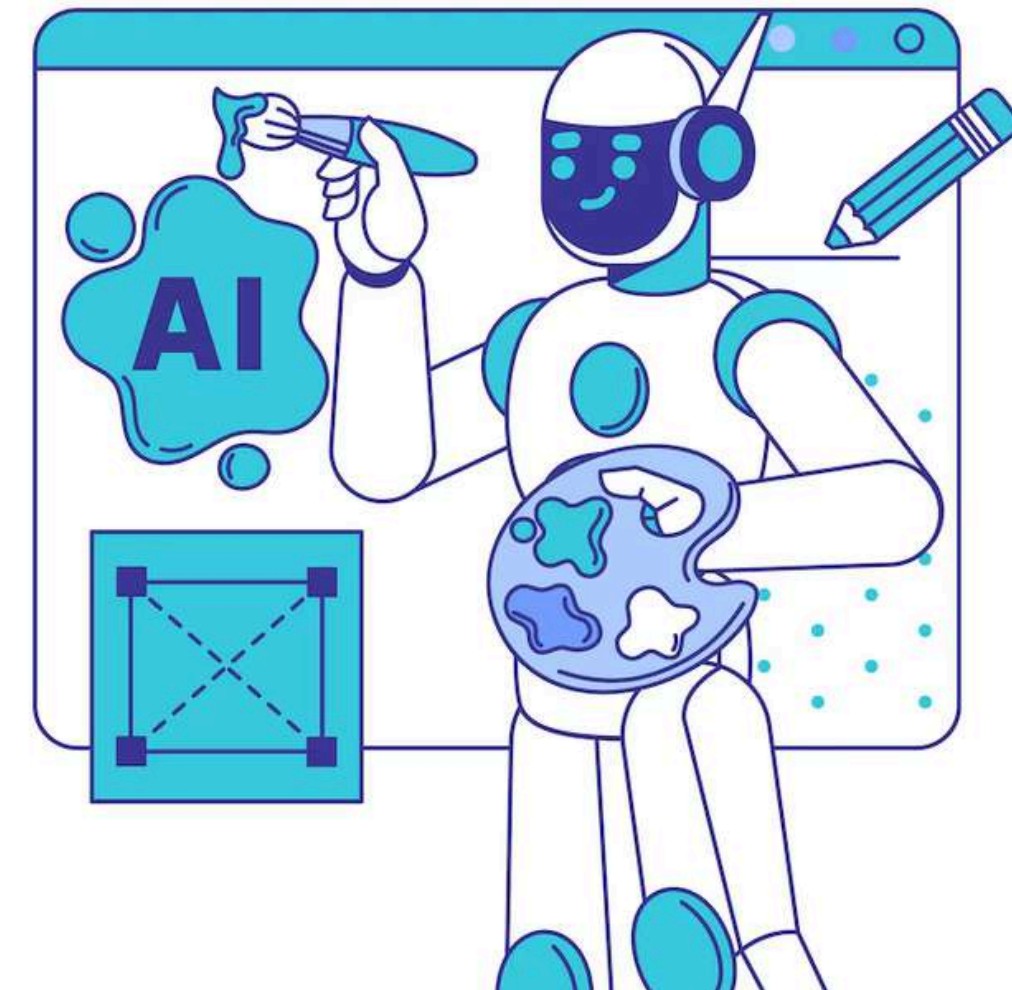
```
max_depth=200,  
min_samples_split=2,  
min_samples_leaf=1,  
criterion='gini',  
class_weight='balanced'
```



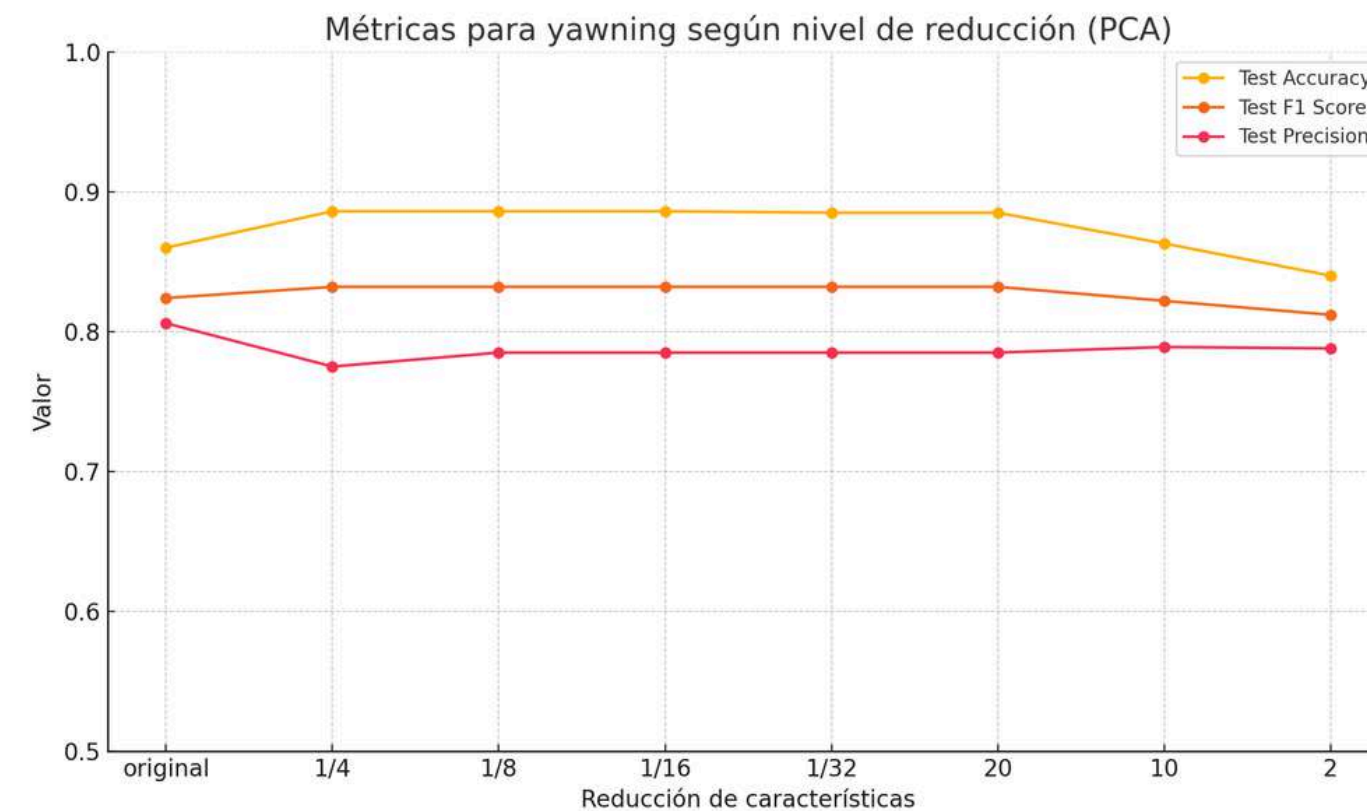
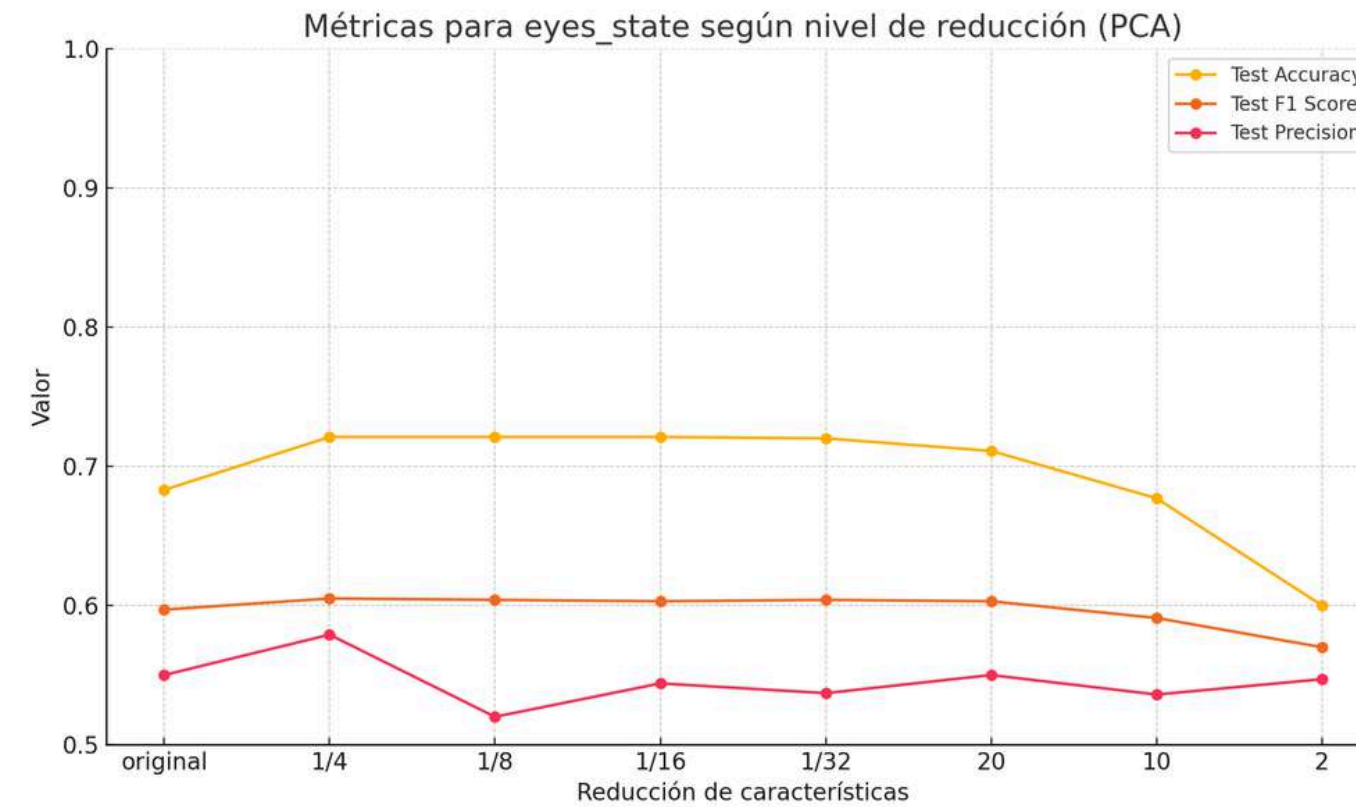
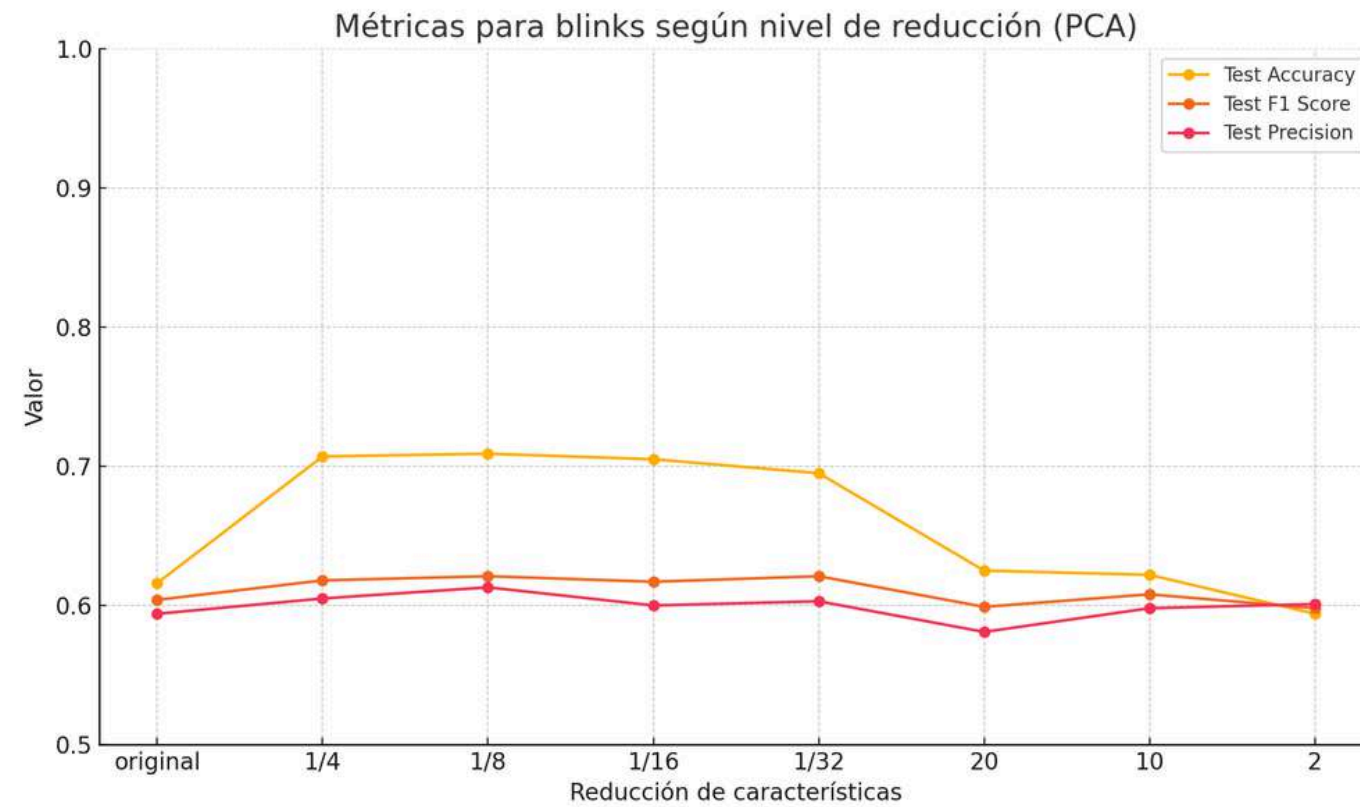
	blinks	eyes_estate	yawning
train_Accuracy	<u>0.980</u>	<u>0.980</u>	<u>0.996</u>
test_Accuracy	<u>0.616</u>	<u>0.683</u>	<u>0.860</u>
train_Precision	<u>0.981</u>	<u>0.981</u>	<u>0.996</u>
test_Precision	<u>0.594</u>	<u>0.550</u>	<u>0.806</u>
train_Recall	<u>0.980</u>	<u>0.980</u>	<u>0.996</u>
test_Recall	<u>0.616</u>	<u>0.683</u>	<u>0.860</u>
train_F1_Score	<u>0.980</u>	<u>0.980</u>	<u>0.996</u>
test_F1_Score	<u>0.604</u>	<u>0.597</u>	<u>0.824</u>
train_Cohen_Kappa	<u>0.957</u>	<u>0.963</u>	<u>0.982</u>
test_Cohen_Kappa	<u>-0.015</u>	<u>-0.005</u>	<u>0.004</u>

Reducción de Dimensiones: PCA/TSNE

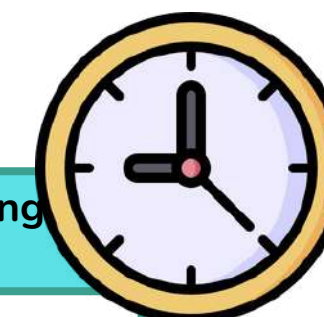
- Se utilizaron estas técnicas para **reducir dimensionalidad** de nuestro dataset.
- En nuestro caso, estas herramientas son **importantísimas**, porque tenemos muchas features, **que suponen mucho costo computacional**, y no todas ellas son útiles para la clasificación; como se verá, hay muchas que generan ruido



Evolución por número de componentes



Reducción de Dimensiones: PCA/TSNE



- Con esta información, concluimos que la mejor reducción está alrededor de 1/32 de las componentes originales.
- En este punto, no perdemos mucha eficiencia en la clasificación, pero sí ganamos algo muy importante: **el tiempo para ejecutar el entrenamiento es significativamente menor**

	blinks	eyes_estate	yawning
train_Accuracy	<u>0.980</u>	<u>0.980</u>	<u>0.996</u>
test_Accuracy	<u>0.695</u>	<u>0.720</u>	<u>0.885</u>
train_Precision	<u>0.981</u>	<u>0.981</u>	<u>0.996</u>
test_Precision	<u>0.603</u>	<u>0.537</u>	<u>0.785</u>
train_Recall	<u>0.980</u>	<u>0.980</u>	<u>0.996</u>
test_Recall	<u>0.695</u>	<u>0.720</u>	<u>0.885</u>
train_F1_Score	<u>0.980</u>	<u>0.980</u>	<u>0.996</u>
test_F1_Score	<u>0.621</u>	<u>0.604</u>	<u>0.832</u>
train_Cohen_Kappa	<u>0.957</u>	<u>0.963</u>	<u>0.982</u>
test_Cohen_Kappa	<u>0.004</u>	<u>0.000</u>	<u>0.000</u>

¿Y el TSNE?

(El mejor obtenido con TSNE: en 1/32)

- En el TSNE, las métricas bajan **considerablemente con respecto al PCA**
- Esto se puede atribuir a que el TSNE **no preserva la estructura global del espacio de características**, solo su distribución
- En nuestro problema, la distribución de los landmarks da lugar a una proyección del TSNE particular (Se verá más adelante) Como la distribución de los landmarks es muy densa y cercana, la hipótesis es que esta nueva distribución complica aún más la clasificación

	blinks	eyes_estate	yawning
train_Accuracy	<u>0.981</u>	<u>0.981</u>	<u>0.996</u>
test_Accuracy	<u>0.573</u>	<u>0.586</u>	<u>0.829</u>
train_Precision	<u>0.982</u>	<u>0.982</u>	<u>0.996</u>
test_Precision	<u>0.589</u>	<u>0.541</u>	<u>0.783</u>
train_Recall	<u>0.981</u>	<u>0.981</u>	<u>0.996</u>
test_Recall	<u>0.573</u>	<u>0.586</u>	<u>0.829</u>
train_F1_Score	<u>0.981</u>	<u>0.981</u>	<u>0.996</u>
test_F1_Score	<u>0.580</u>	<u>0.561</u>	<u>0.805</u>
train_Cohen_Kappa	<u>0.959</u>	<u>0.965</u>	<u>0.984</u>
test_Cohen_Kappa	<u>-0.026</u>	<u>-0.013</u>	<u>-0.041</u>



Reducción de Dimensiones: Otros Algoritmos

- **Truncated SVD**

Técnica de reducción de dimensionalidad que conserva los componentes más importantes sin centrar los datos.

- Reduce dimensiones como PCA, pero es mejor para matrices dispersas (como TF-IDF).
- No necesita centrar, funciona con datos dispersos y es más eficiente en gran escala.

	blinks	eyes_estate	yawning
train_Accuracy	<u>0.980</u>	<u>0.980</u>	<u>0.996</u>
test_Accuracy	<u>0.692</u>	<u>0.720</u>	<u>0.886</u>
train_Precision	<u>0.981</u>	<u>0.981</u>	<u>0.996</u>
test_Precision	<u>0.593</u>	<u>0.528</u>	<u>0.785</u>
train_Recall	<u>0.980</u>	<u>0.980</u>	<u>0.996</u>
test_Recall	<u>0.692</u>	<u>0.720</u>	<u>0.886</u>
train_F1_Score	<u>0.980</u>	<u>0.980</u>	<u>0.996</u>
test_F1_Score	<u>0.616</u>	<u>0.604</u>	<u>0.832</u>
train_Cohen_Kappa	<u>0.957</u>	<u>0.963</u>	<u>0.982</u>
test_Cohen_Kappa	<u>-0.009</u>	<u>0.001</u>	<u>-0.000</u>

Reducción de Dimensiones: Otros Algoritmos

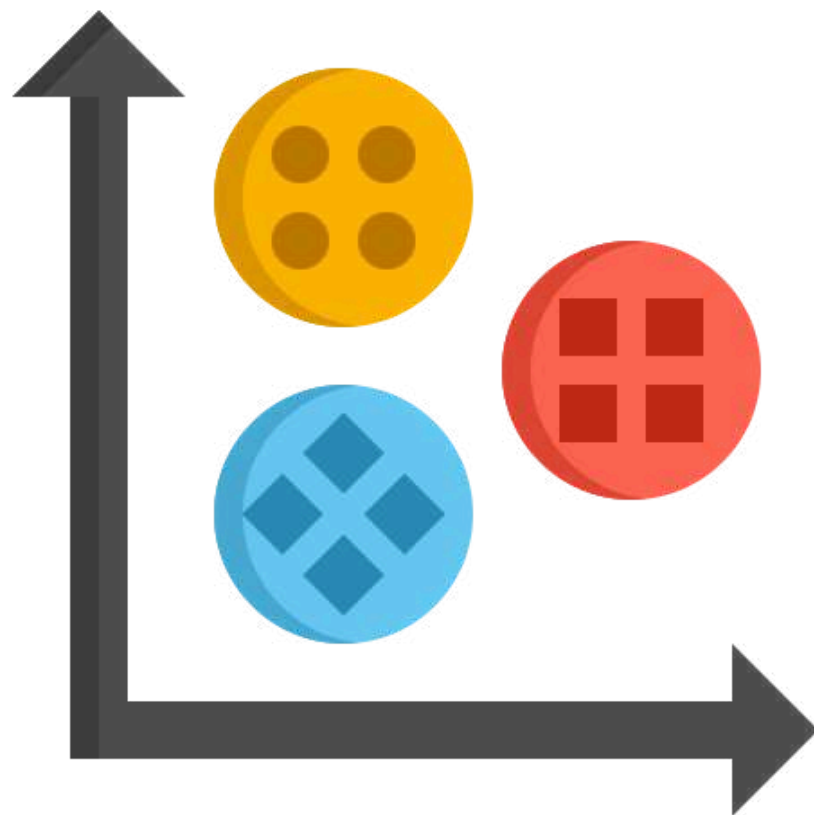
- **Factor Analysis**

Técnica de reducción que modela la variabilidad común entre variables como factores latentes.

- A diferencia de PCA, se enfoca en las relaciones ocultas, no en la varianza total.
- Útil para identificar estructuras subyacentes, pero menos eficiente con datos dispersos.

	blinks	eyes_estate	yawning
train_Accuracy	<u>0.980</u>	<u>0.980</u>	<u>0.996</u>
test_Accuracy	<u>0.702</u>	<u>0.718</u>	<u>0.886</u>
train_Precision	<u>0.981</u>	<u>0.981</u>	<u>0.996</u>
test_Precision	<u>0.605</u>	<u>0.535</u>	<u>0.785</u>
train_Recall	<u>0.980</u>	<u>0.980</u>	<u>0.996</u>
test_Recall	<u>0.702</u>	<u>0.718</u>	<u>0.886</u>
train_F1_Score	<u>0.980</u>	<u>0.980</u>	<u>0.996</u>
test_F1_Score	<u>0.620</u>	<u>0.604</u>	<u>0.832</u>
train_Cohen_Kappa	<u>0.957</u>	<u>0.963</u>	<u>0.982</u>
test_Cohen_Kappa	<u>0.006</u>	<u>-0.001</u>	<u>-0.001</u>

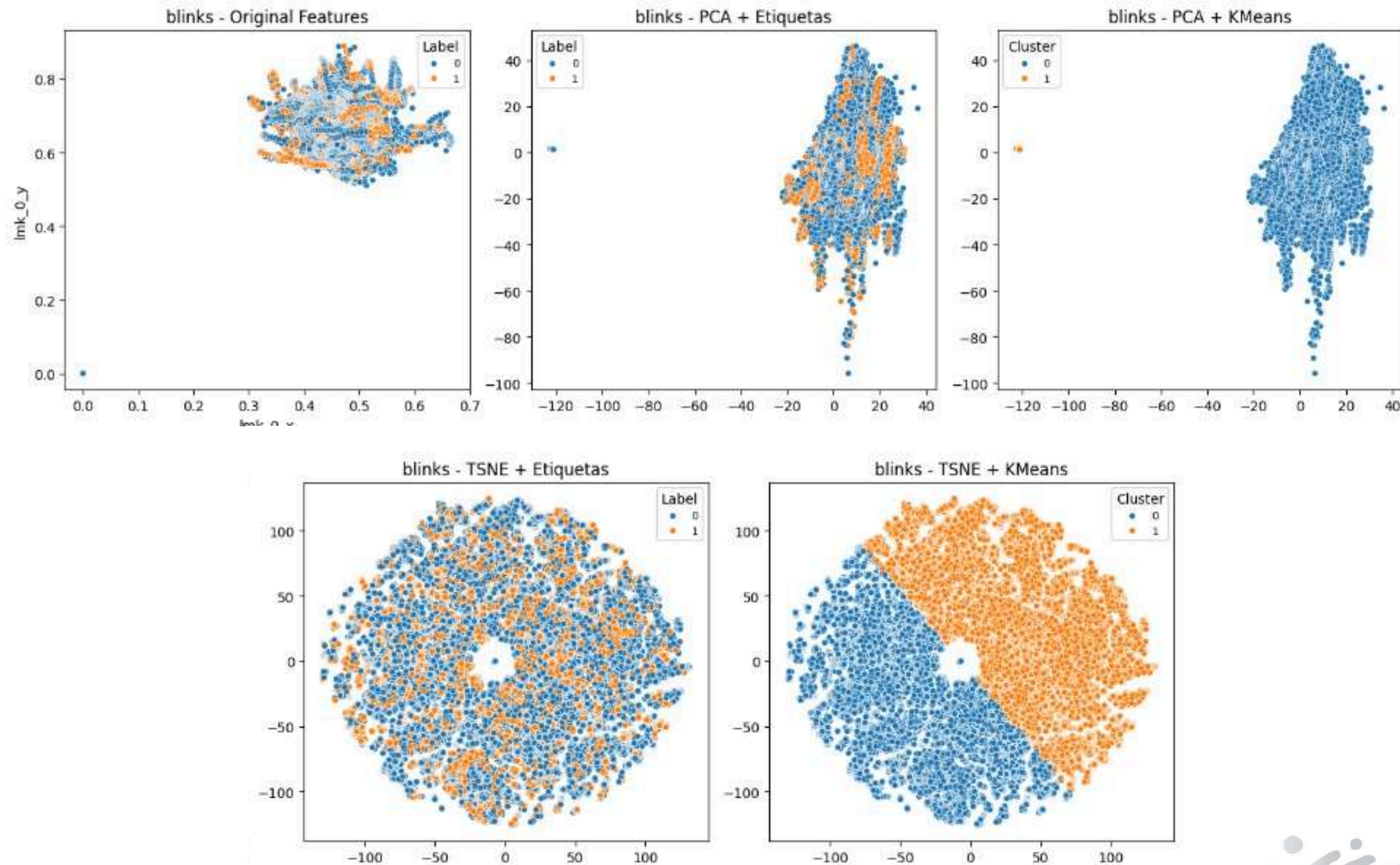
Métodos de Agrupamiento: K-Means, DBSCAN y Aglomerative Clustering



- Ahora, vamos a intentar encontrar si, tras aplicar técnicas de reducción de componentes, **los algoritmos de clustering son capaces de diferenciar correctamente los grupos del dataset**
- Por ello, vamos a partir de una representación visual en **2 componentes**: una generada por un PCA, y otro por un TSNE

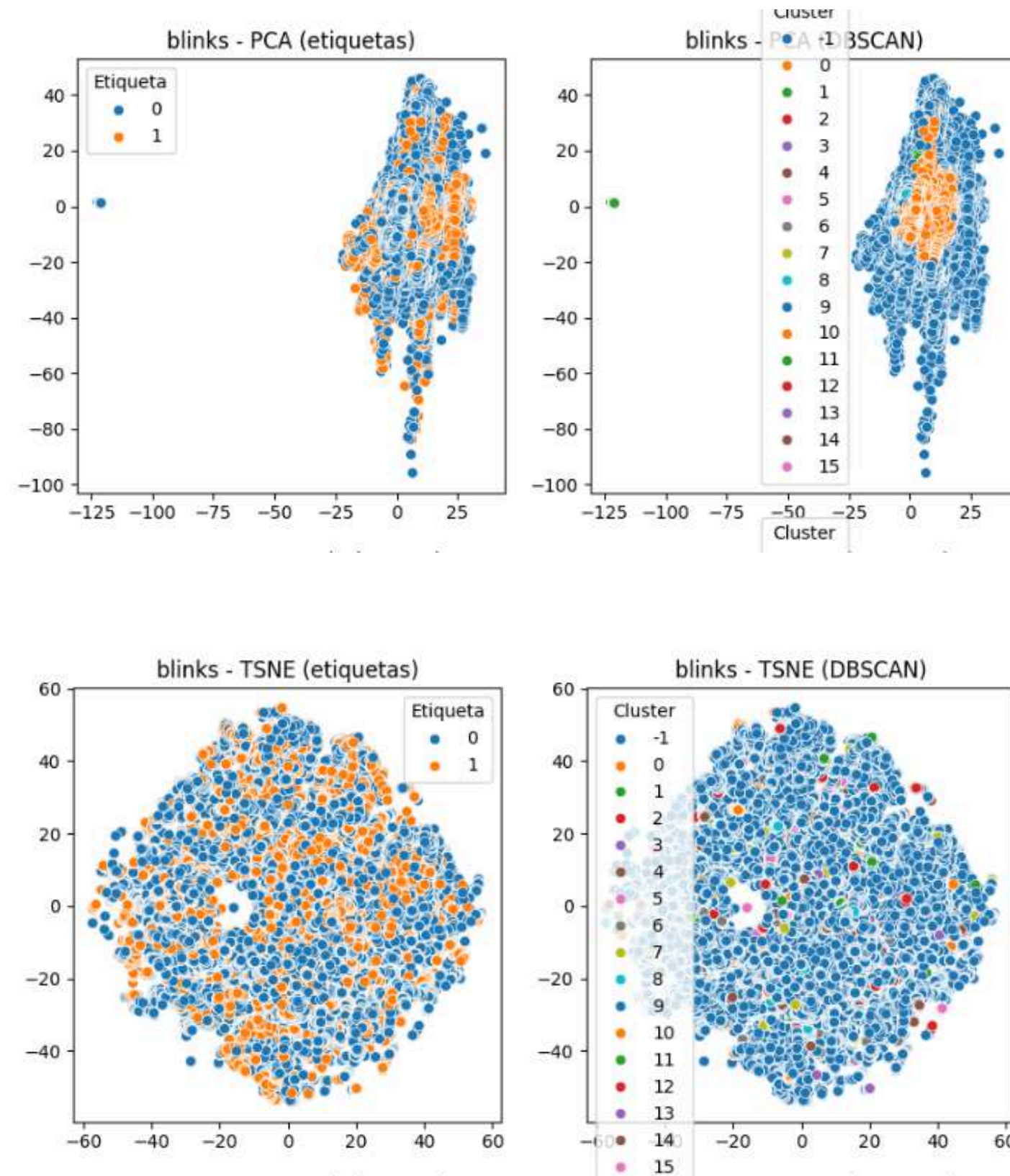
K-Means

Comparación de Clustering por Target



DBSCAN

- Con DBSCAN, los resultados son muy variables dependiendo de los hiperpárametros. Se probó con variaciones de ϵ y min_samples , pero los resultados de separación son genuinamente irrelevantes.
- (Con un ejemplo de $\text{min_samples} = 50$), $\epsilon = 0.5$



Otros algoritmos de clustering

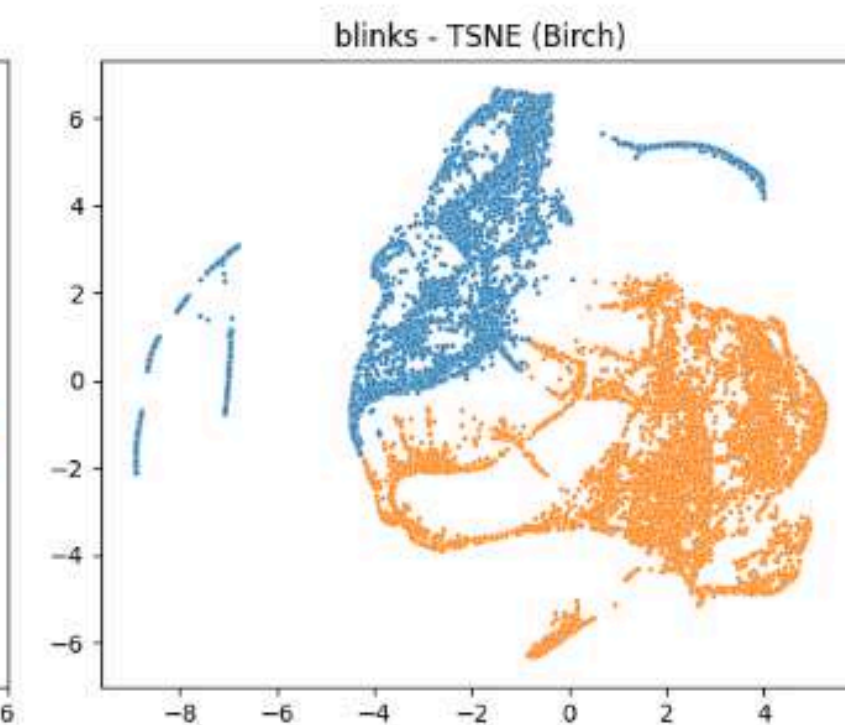
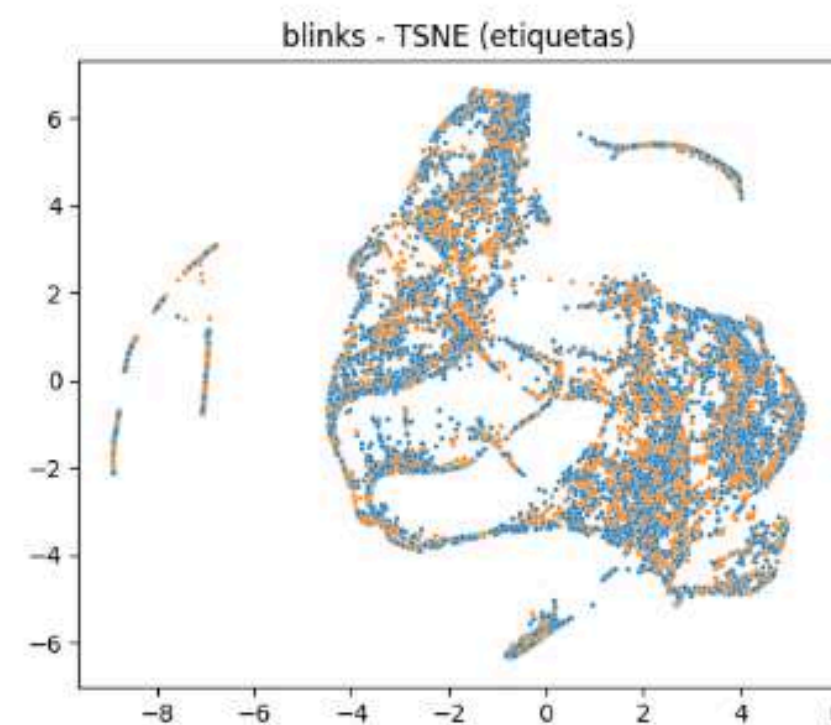
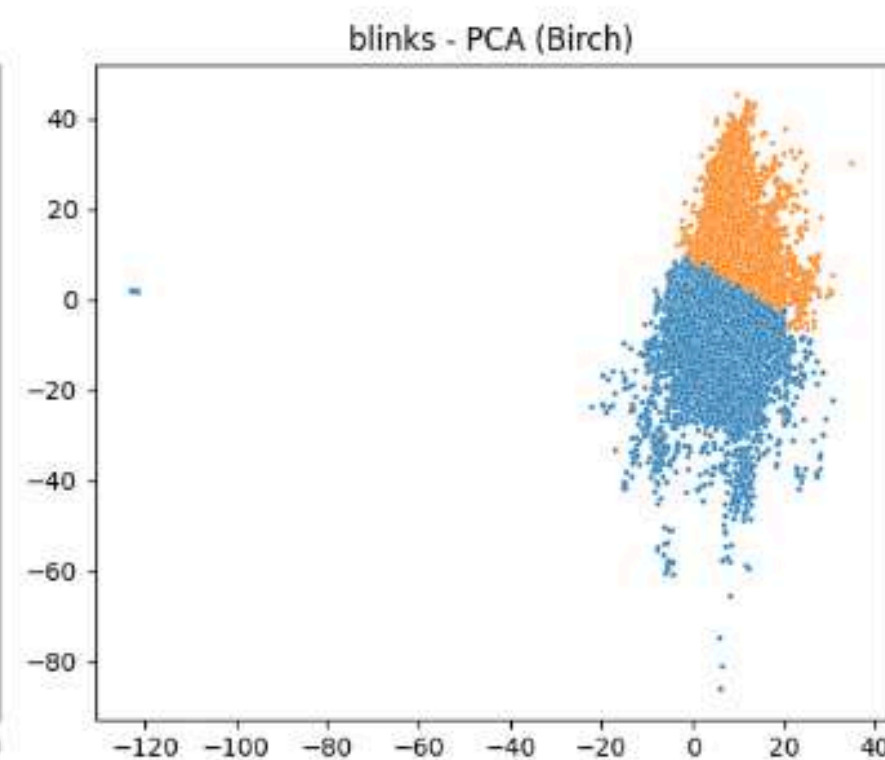
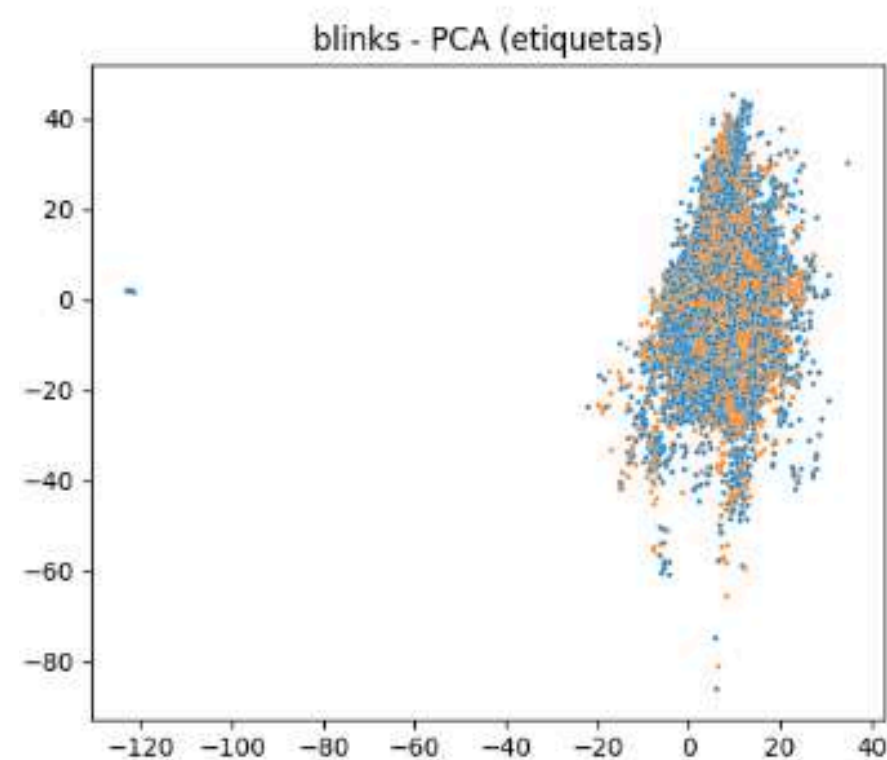
BIRCH

- Construye un árbol de características (**CF Tree**) que resume los datos en pequeñas subestructuras llamadas clústeres subyacentes.
- Agrupa datos de forma jerárquica y eficiente
- Es bueno para grandes cantidades de datos por esa misma razón.

The BIRCH clustering algorithm consists of two stages:



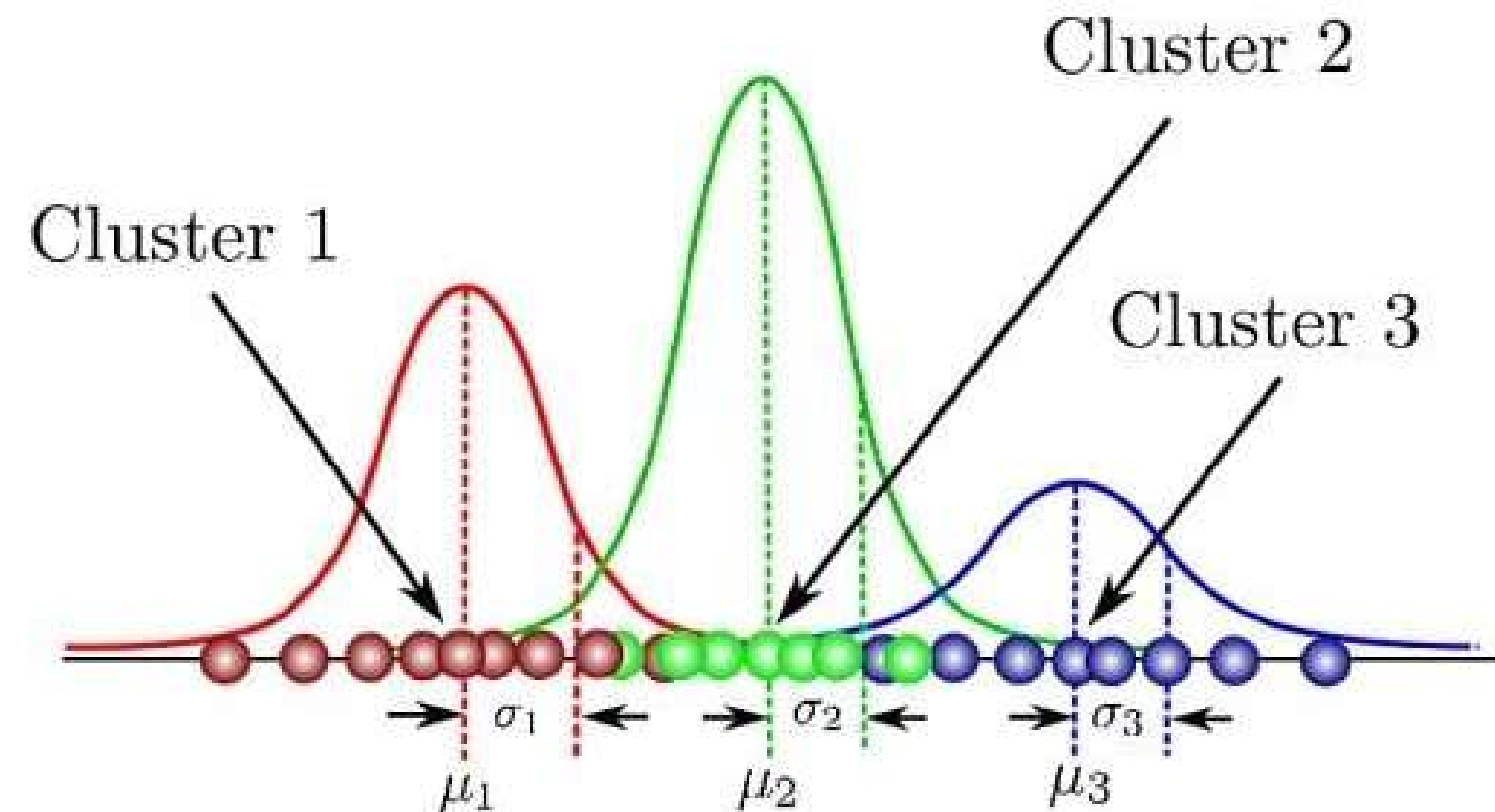
BIRCH



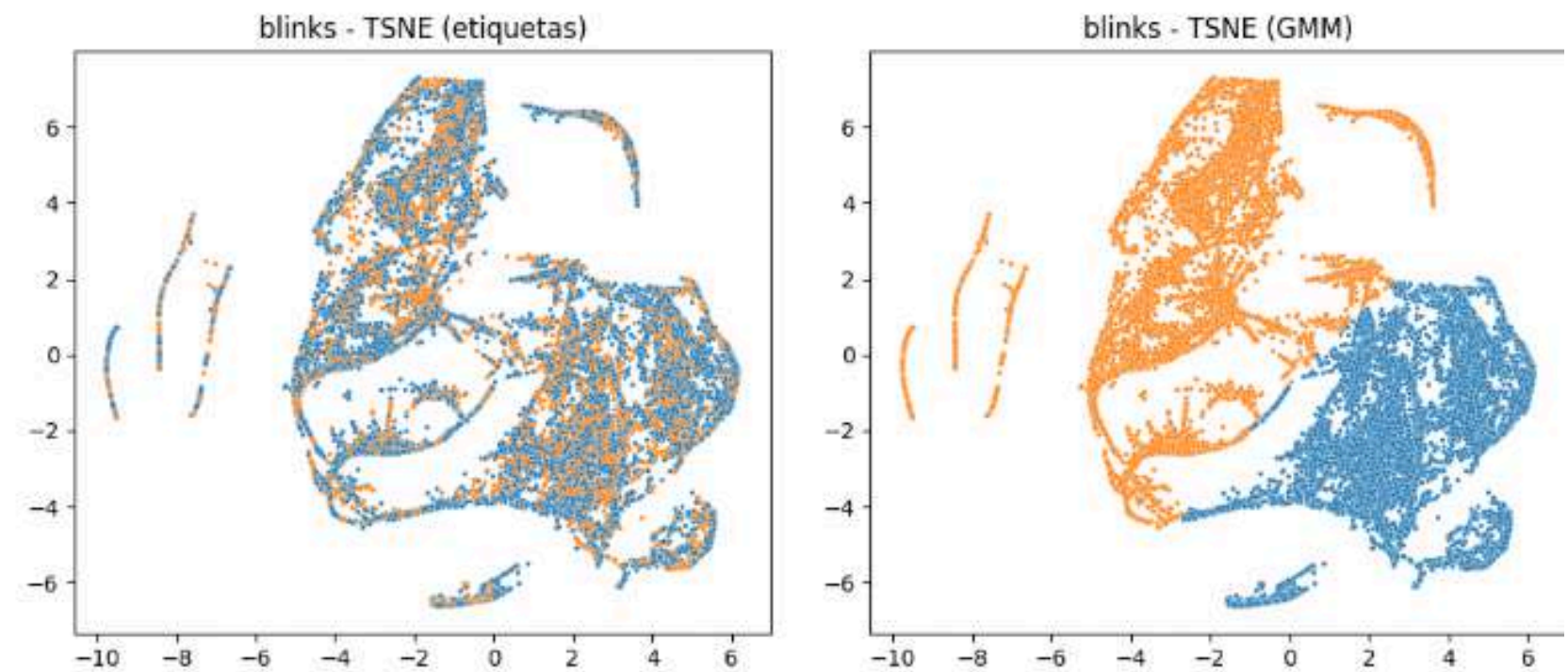
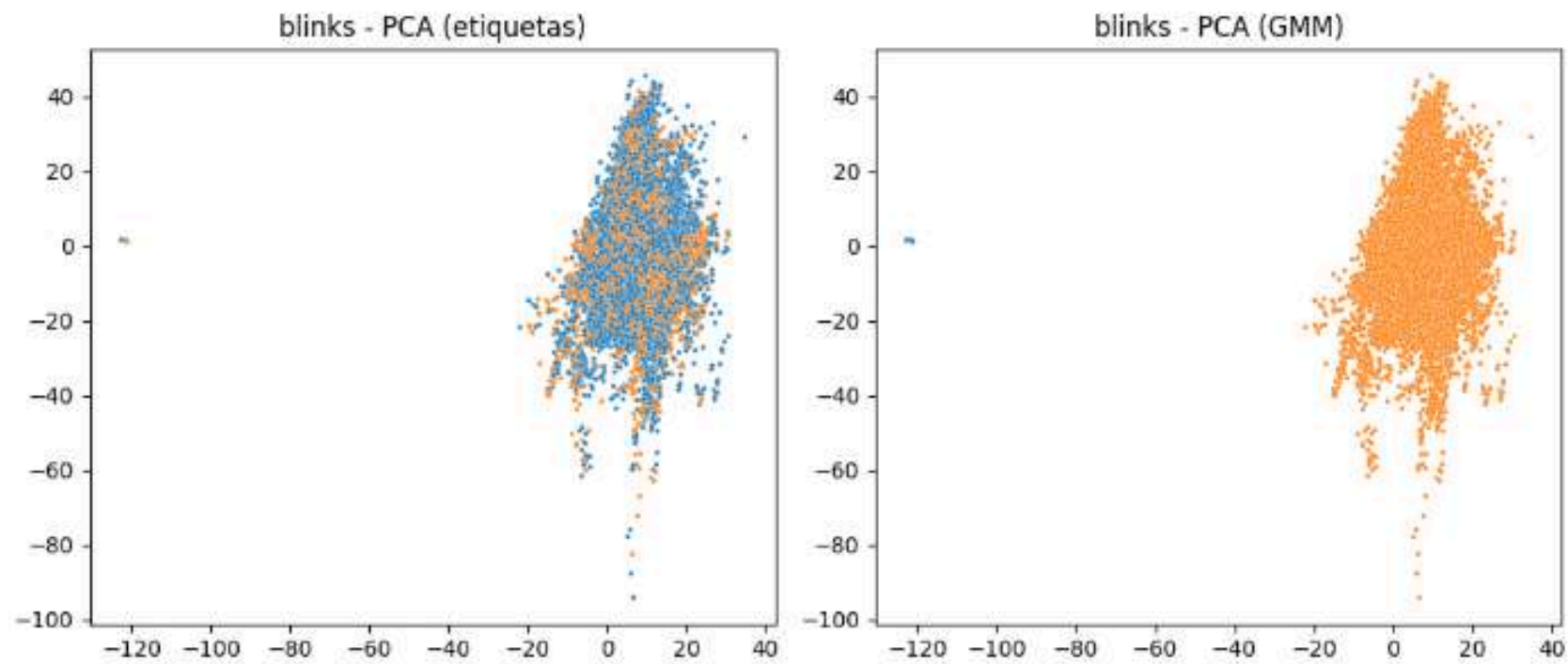
Otros algoritmos de clustering

Gaussian Mixture

- Se basa en modelos estadísticos, asumiendo que los datos provienen de una combinación de varias distribuciones normales (gaussianas).
- Es probabilístico, lo que significa que asigna a cada punto una probabilidad de pertenecer a cada clúster.
- Se itera con el algoritmo **Expectation-Maximization**

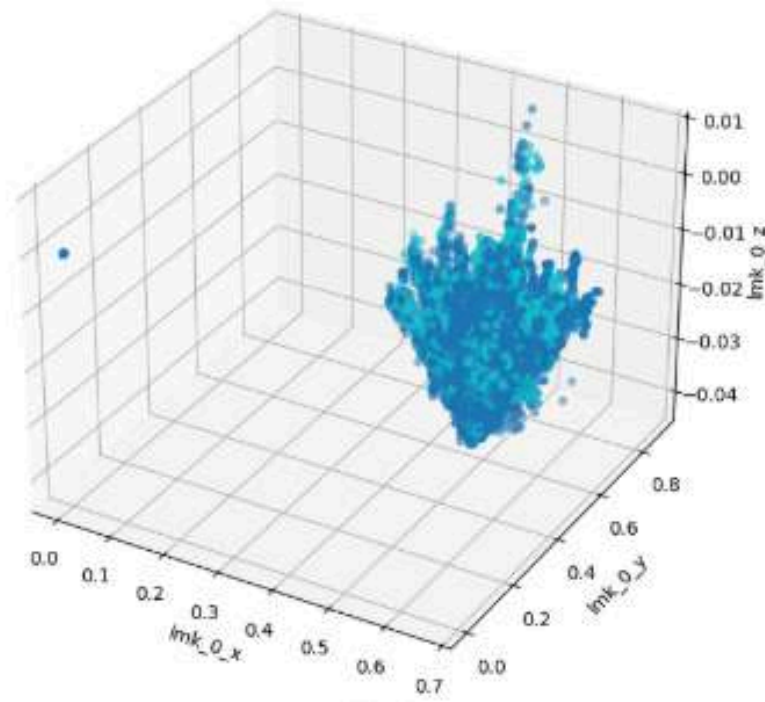


Gaussian Mixture

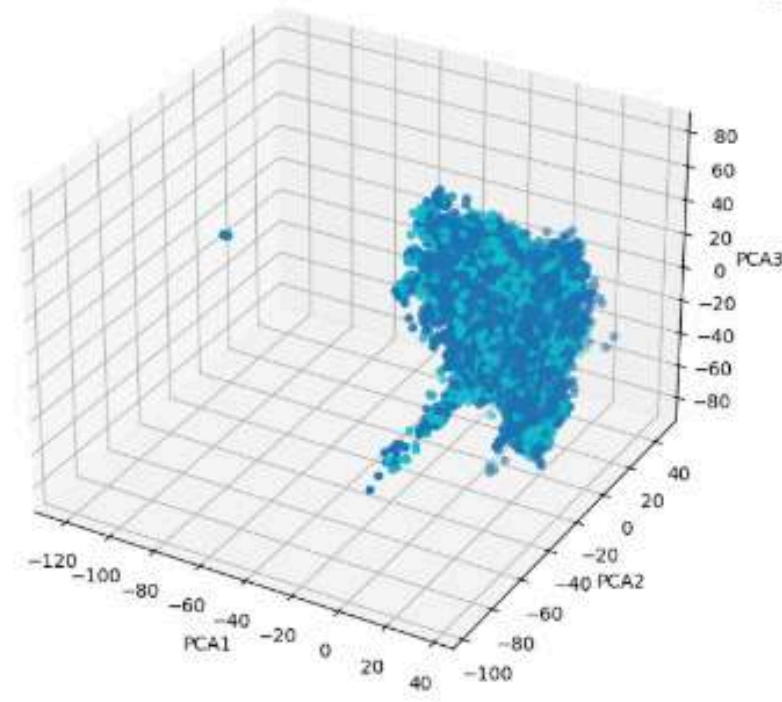


Bonus: K-Means en 3D

blinks - Original Features

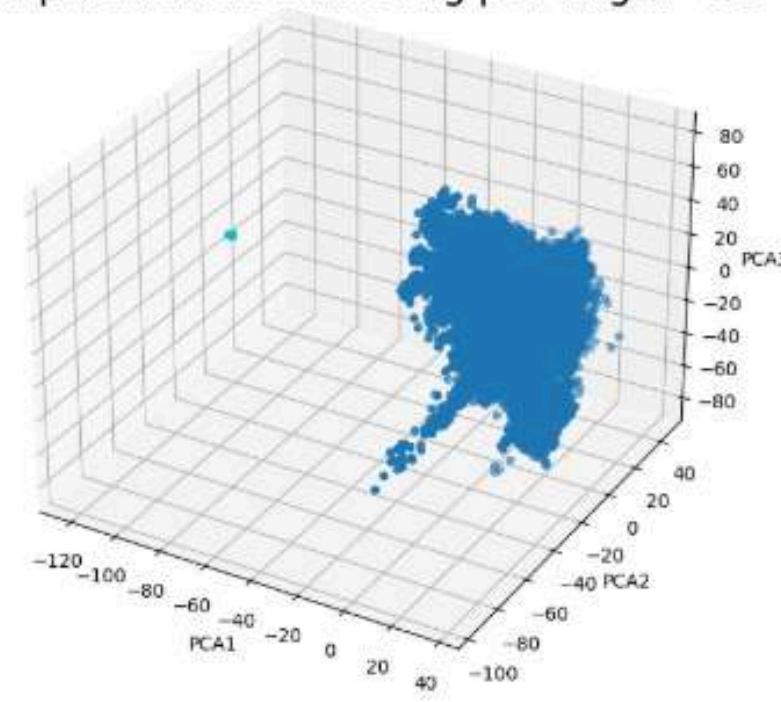


blinks - PCA + Etiquetas

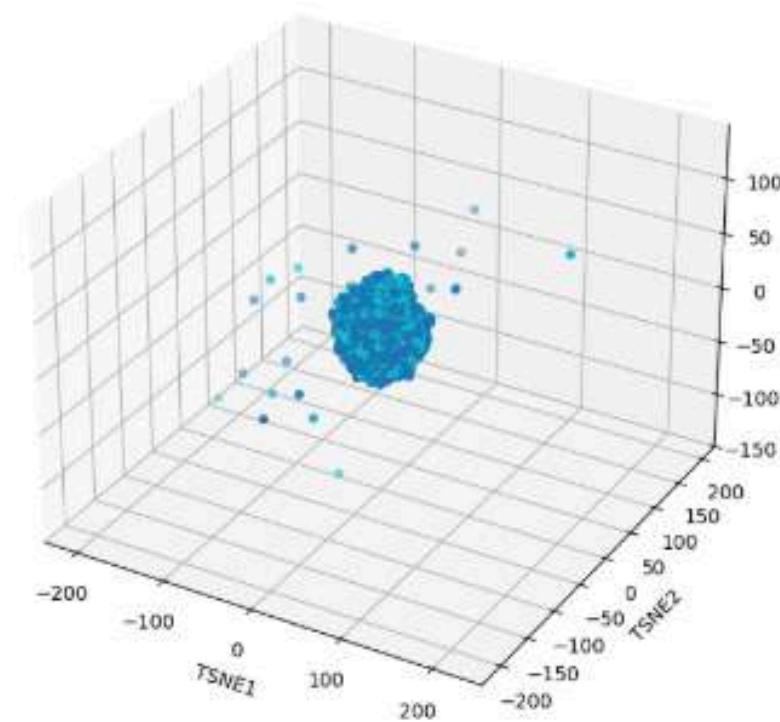


blinks - PCA + KMeans

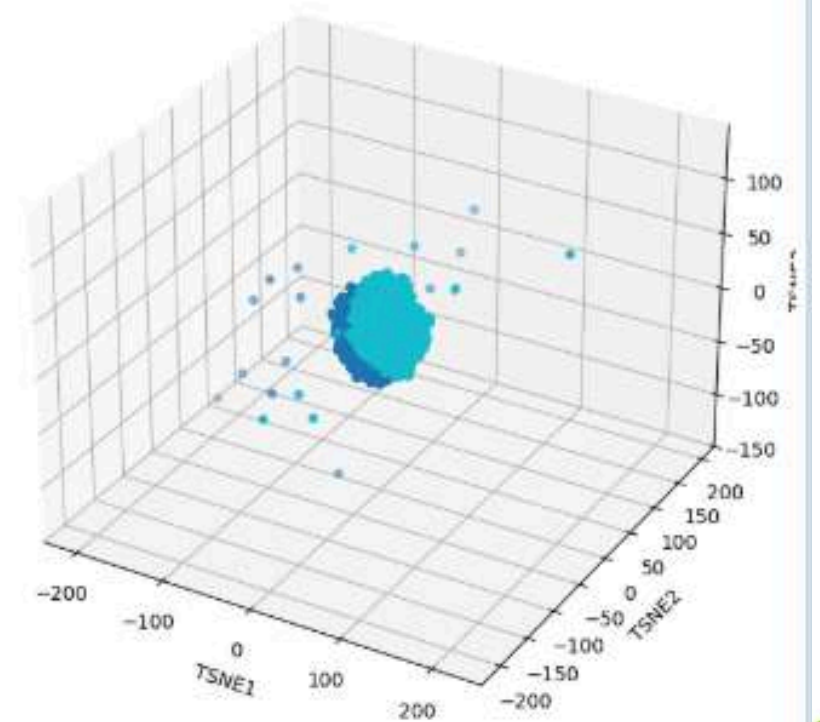
Comparación de Clustering por Target - 3D



blinks - TSNE + Etiquetas

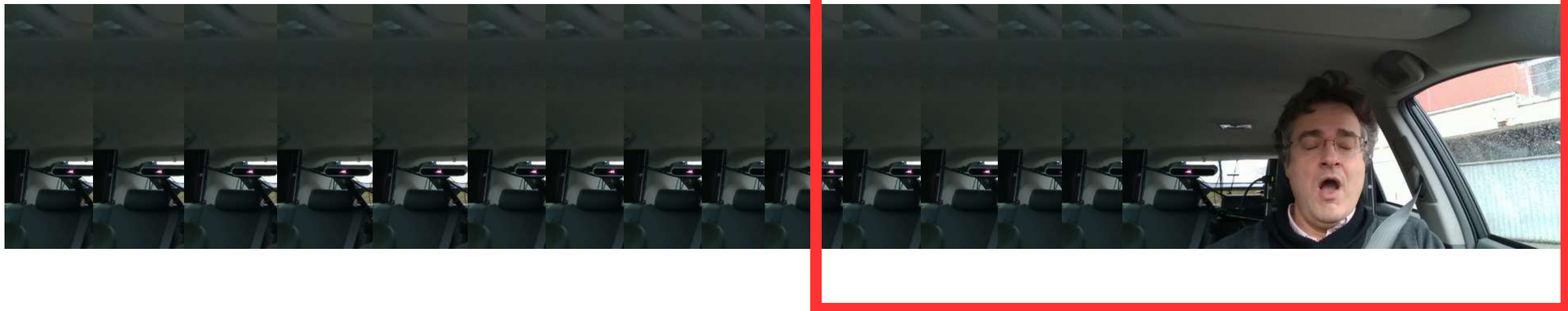


blinks - TSNE + KMeans

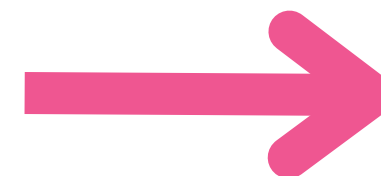


Uso de las predicciones

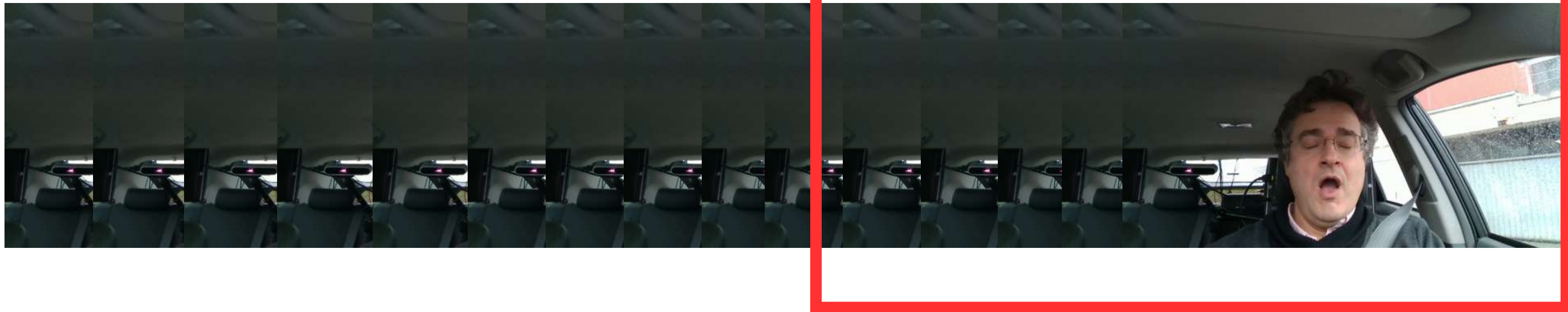
- Observar ventanas con predicciones de las etiquetas, y asignar cierta **tolerancia** que dependa de ellas para mandar una **alerta**



$$w0 * \text{blinks_window} + w1 * \text{eyes_state_window} + w2 * \text{yawning_window} > \text{tolerancia_value}$$



Uso de las predicciones



- Usando:
- `ventana_n_frames = 90`
- `w0, w1, w2 = 0.3, 1.2, 1.5`
- `tolerancia_value = 60`

$$w0 * \text{blinks_window} + w1 * \text{eyes_state_window} + w2 * \text{yawning_window} > \text{tolerancia_value}$$



Resultados (videos)



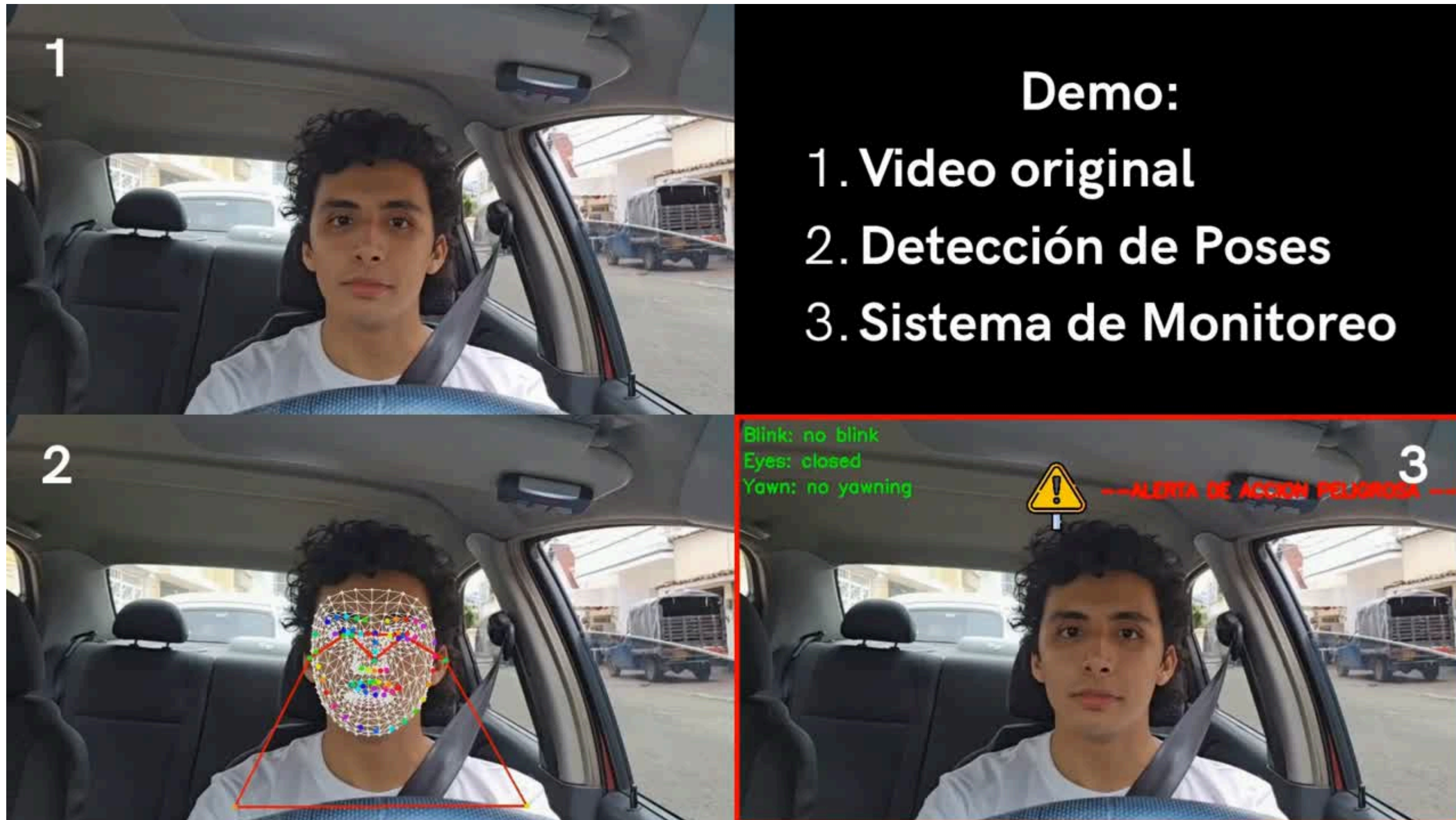
Resultados (videos)



Resultados (videos)



Conclusiones Finales



Limitaciones y Trabajo Futuro

- El sistema tiene un rendimiento aceptable cuando se evalúan las grabaciones procedentes del DMD, pero se observa que está ligado a la posición relativa de la cámara con respecto al conductor; al grabar desde otro ángulo, las predicciones disminuyen su exactitud.
- Para solucionar esto, se podría considerar aplicar otras transformaciones y preprocesado del dataset, antes de pasar al aprendizaje.



Limitaciones y Trabajo Futuro

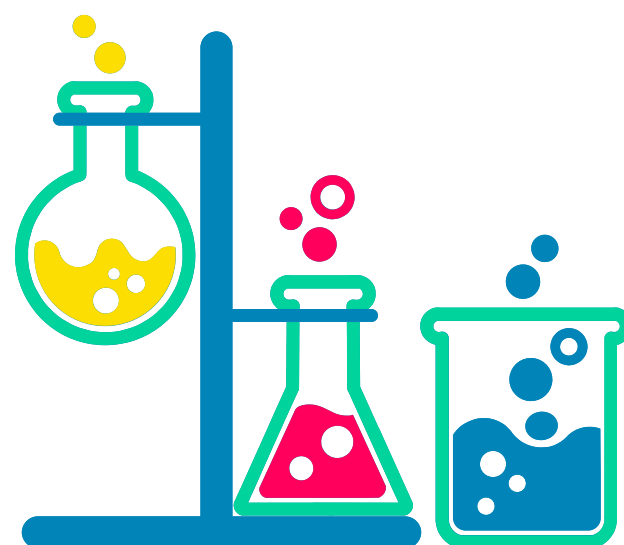
- Se intentó realizar una detección en tiempo real de las posturas para su procesamiento inmediato. No obstante, se comprobó que esta implementación no es posible desde la aproximación del proyecto: mientras la detección de Mediapipe es prácticamente instantánea, las inferencias de poses de ViTPose son muy lentas y dan lugar a latencias muy elevadas.
- Como trabajo futuro, sería llamativo explorar otras divisiones del dataset (Gaze y Distraction)



Prueba rápida: Gaze y Distraction

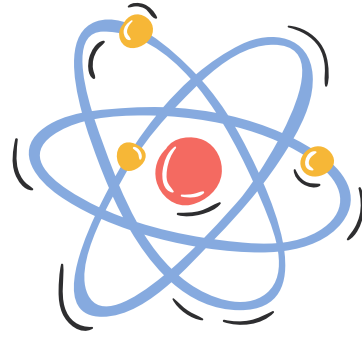
Gaze

- 12 Sujetos: **9 train y 3 test**
- 1 Feature a predecir: **'gaze_zone'** (8 clases)
- **60723 filas x 1451 columnas**



Distraction

- 6 Sujetos: **5 train y 1 test**
- 4 Feature a predecir:
'gaze_on_road' (2 clases),
'driver_actions' (14 clases),
'hands_using_wheel' (4 clases),
'talking' (2 clases)
- **141750 filas x 1455 columnas**



Gaze

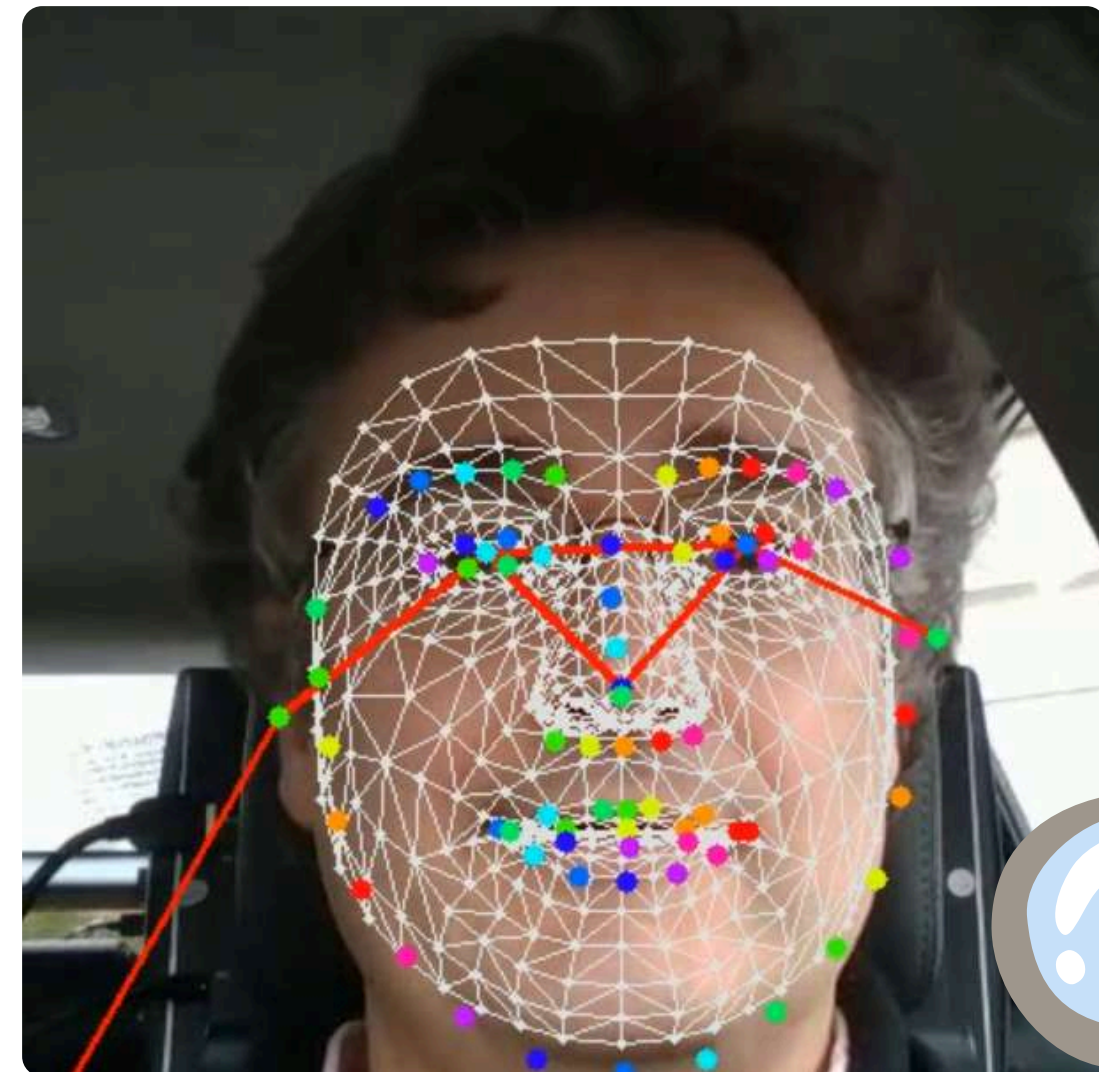
	blinks
train_Accuracy	<u>0.941</u>
test_Accuracy	<u>0.271</u>
train_Precision	<u>0.956</u>
test_Precision	<u>0.249</u>
train_Recall	<u>0.941</u>
test_Recall	<u>0.271</u>
train_F1_Score	<u>0.944</u>
test_F1_Score	<u>0.233</u>
train_Cohen_Kappa	<u>0.933</u>
test_Cohen_Kappa	<u>0.175</u>

Distraction

	driver_actions	gaze_on_road	hands_using_wheel	talking
train_Accuracy	<u>1.000</u>	<u>1.000</u>	<u>1.000</u>	<u>1.000</u>
test_Accuracy	<u>0.321</u>	<u>0.858</u>	<u>0.452</u>	<u>0.838</u>
train_Precision	<u>1.000</u>	<u>1.000</u>	<u>1.000</u>	<u>1.000</u>
test_Precision	<u>0.190</u>	<u>0.776</u>	<u>0.416</u>	<u>0.703</u>
train_Recall	<u>1.000</u>	<u>1.000</u>	<u>1.000</u>	<u>1.000</u>
test_Recall	<u>0.321</u>	<u>0.858</u>	<u>0.452</u>	<u>0.838</u>
train_F1_Score	<u>1.000</u>	<u>1.000</u>	<u>1.000</u>	<u>1.000</u>
test_F1_Score	<u>0.214</u>	<u>0.794</u>	<u>0.383</u>	<u>0.765</u>
train_Cohen_Kappa	<u>1.000</u>	<u>1.000</u>	<u>1.000</u>	<u>1.000</u>
test_Cohen_Kappa	<u>0.048</u>	<u>0.002</u>	<u>0.060</u>	<u>-0.001</u>

Conclusiones

- Las herramientas de extracción de posturas a través de videos (en este caso, Mediapipe FaceMesh y ViTPose), probaron ser un buen punto de partida para optar por un enfoque tabular en el análisis de videos que involucren ciertos comportamientos humanos (en este caso, aquellos relacionados al volante).



Conclusiones

- Distintos modelos de ML tradicionales probaron ser efectivos a la hora de llevar a cabo la tarea de clasificación; destacando, entre ellos, la labor del RandomForestClassifier y el KNeighbors Classifier. Se puede atribuir el mejor rendimiento de estos últimos a la naturaleza de coordenadas de los datos y cómo estas se agrupan entre sí.



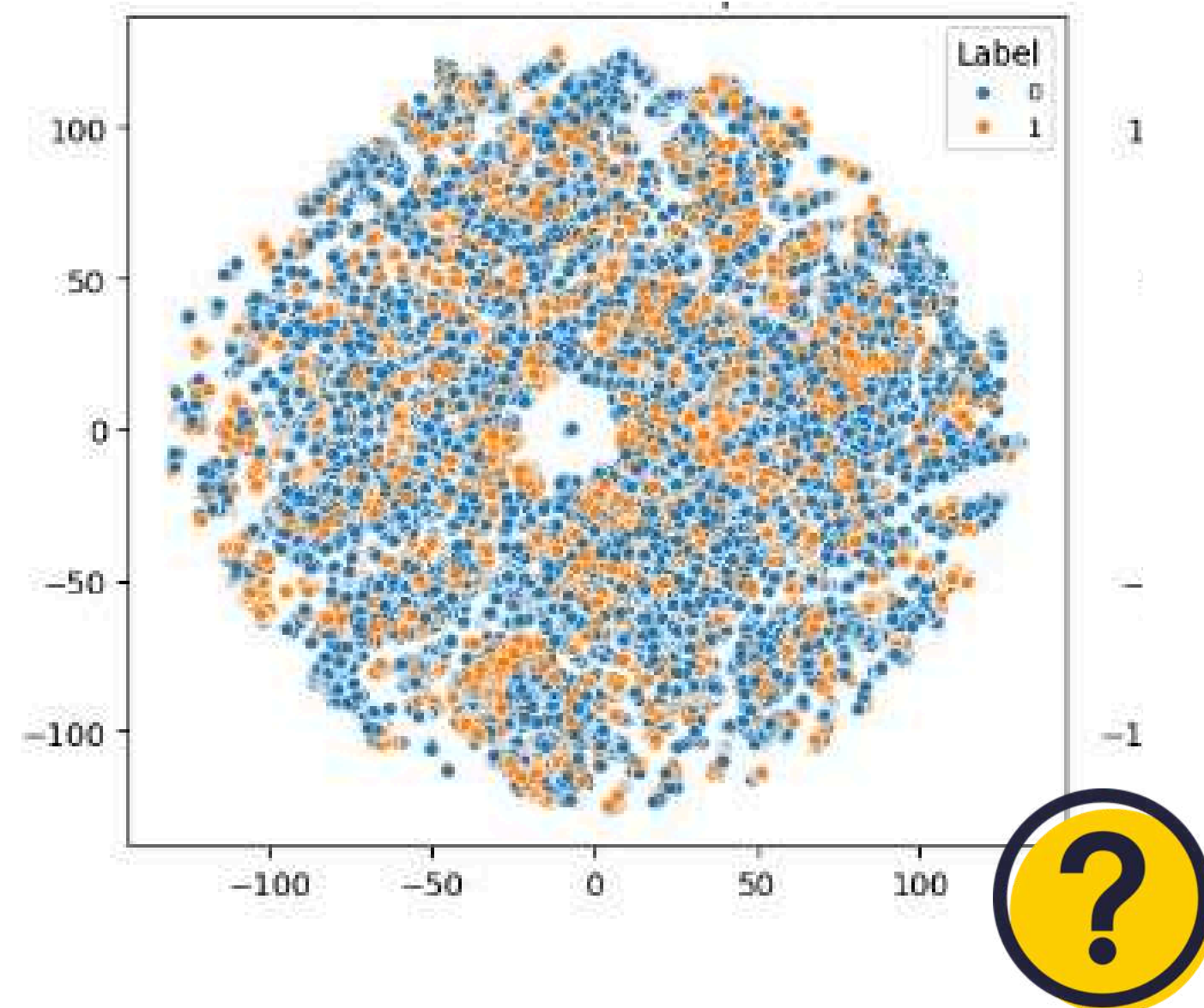
Conclusiones

- Destaca la labor del aprendizaje no supervisado a través de los métodos de reducción de dimensionalidad. Específicamente, el PCA (Análisis de Componentes Principales) permitió mejorar el rendimiento y las métricas a través de la eliminación del ruido y el enfoque los componentes con mayor varianza.



Conclusiones

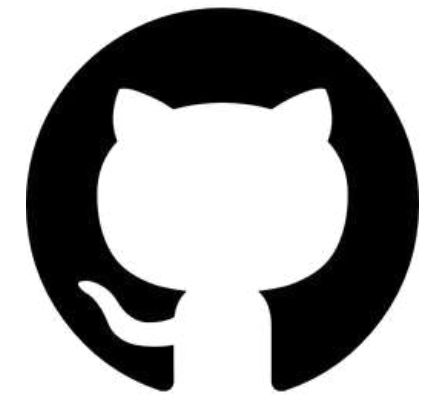
- En cambio, los métodos de clustering implementados no fueron efectivos en la diferenciación de las clases. Un análisis visual a través de PCA o t-SNE permite observar que la separación de las etiquetas no es una labor trivial para nuestro caso de estudio.



Repositorio

Repositorio

- https://github.com/popcorner893/Sistema_Monitoreo_Conductor



¡Muchas
GRACIAS!



¡Gracias!